

strands-aerospike benchmark report

Environment:

- Host: Linux-7.0.0-1012-aws-x86_64-with-glibc2.43, Python 3.12.14
- Aerospike: Community Edition, single-node Docker container started by `./scripts/start_aerospike_ce.sh -- docker exec strands-aerospike-ce asinfo -v build` reports build info per node: 8.1.2.4-4.
- S3Storage / S3SessionManager: benchmarked against a local `moto[server]` `ThreadedMotoServer` (real local HTTP server implementing the S3 API), **not** real AWS S3. This measures real HTTP + (de)serialization overhead but *not* real network latency to an AWS region -- treat the S3 numbers here as a floor, not a prediction of production S3 latency.
- FileSessionManager / LocalFileStorage / FileMemoryStore: local filesystem (the scratch directory this run created and deleted afterward), not a network filesystem.
- **These are single-node, single-machine, local-Docker numbers.** They are not production-cluster numbers for any backend; treat them as relative comparisons under identical local conditions, not absolute latency predictions for a real deployment.
- BedrockKnowledgeBase (the only other in-repo MemoryStore) requires a live AWS Bedrock Knowledge Base and cannot be exercised locally, so it is excluded rather than faked.
- **Redis/Valkey, PostgreSQL, and MongoDB were explicitly checked for** as candidate comparison backends for the Storage, SessionRepository, and MemoryStore interfaces (strands-agents's own dependency metadata and source tree were searched for a Redis/Valkey-backed, SQL/Postgres-backed, or Mongo-backed implementation of any of the three). None exist in strands-agents today -- its only shipped backends for these interfaces are the in-memory, local-filesystem, S3, and (for MemoryStore) Bedrock Knowledge Base implementations benchmarked here. This comparison set is therefore exhaustive against what strands-agents currently ships, not a partial selection.
- **Bulk-scale tier:** the Storage interface's largest scale targets -- `bulk-target-gb` (10 GB requested, 10.00GB actual at 512KiB/value) of total value bytes. InMemoryStorage and S3Storage are excluded from that tier: InMemoryStorage is a plain in-process dict, and S3Storage's moto backend buffers each object under ~5MB in a `SpooledTemporaryFile` that never rolls over to disk at this value size -- both would need to hold the entire bulk dataset in this benchmark process's own RAM. This host has ~7.6GB of RAM total, not enough headroom to do that safely, so those two are compared only at the smaller scales. AerospikeStorage's "test" namespace storage capacity (`STORAGE_GB`) and primary-index memory (`MEM_GB`) were both raised in `scripts/start_aerospike_ce.sh` to give headroom above this target. **This is a deliberate ceiling driven by this host's available disk and RAM, not a hard design limit** -- raise `--bulk-target-gb` (e.g. to 100) once a host with more free disk (and, if InMemoryStorage/S3Storage should stay included at that scale, more free RAM) is available.

Methodology

- **Storage** (AerospikeStorage, InMemoryStorage, LocalFileStorage, S3Storage): scales are pre-existing key counts (100 / 500 / 2,000), bracketing a realistic range for snapshot/plugin-blob storage. Values are ~1.2KB of pseudo-text. `write/read/delete/list` are timed with 100 samples (15 discarded warmup) at every scale. `search` (fetch-every-key token-overlap scoring on every non-Aerospike backend, and on AerospikeStorage itself -- it intentionally doesn't index-optimize this path, see `docs/DESIGN.md`) is inherently $O(n)$ per query; it is timed with a reduced 20 samples, and only at the two smaller scales, to keep total runtime reasonable without skipping the operation. A fourth "bulk" tier is also run, sized so total value bytes hit `--bulk-target-gb` (10GB by default) using 512KiB values instead of the ~1.2KB values used at the smaller scales -- reaching that target via ~1.2KB values would need millions of keys, impractical to seed against file- and HTTP-backed backends, so the bulk tier grows the payload size instead of the key count. `list`'s reps drop to 5 at the bulk tier only, since a full-set scan reads every record's bins off the storage device before filtering, and 100 reps over a multi-gigabyte dataset would mean scanning proportionally more.

- **SessionRepository** (AerospikeSessionManager, FileSessionManager, S3SessionManager, via each backend's own manager instance): scales are pre-existing message counts per agent (20 / 100 / 500), bracketing a long but realistic single-agent conversation. create_message/read_message/ list_messages are timed with 100 samples (15 discarded warmup) at every scale. delete_session is destructive -- each rep must recreate a whole fresh session first -- so it is timed with a reduced 20 samples at the smallest scale only, measuring just the delete_session call itself (the per-rep recreate is unmeasured setup). A fourth "bulk" tier (2000 messages, each padded to ~8KB) is also run -- large enough to meaningfully exceed the smaller tiers, but a single agent conversation reaching the Storage interface's literal multi-gigabyte target is not a realistic shape for this subsystem, so this tier brackets SessionRepository's own realistic ceiling instead (see the Storage bullet above for where that larger target actually applies).
- **MemoryStore** (AerospikeMemoryStore, FileMemoryStore): scales are pre-existing entry counts (50 / 250 / 1,000), bracketing a substantial agent-memory store. add, search (a common query token matching every seeded entry -- worst case for a full-scan backend, but also worst case for an index-backed lookup, since the index can't narrow a query every record matches), and search_selective (a store-unique rare token matching exactly one seeded entry regardless of scale -- the case an index-backed lookup is actually built for) are all timed with 100 samples (15 discarded warmup) at every scale. A fourth "bulk" tier (5000 entries, each padded with ~4KB of extra body text) is also run, sized to a large but realistic single memory/knowledge namespace rather than the Storage interface's literal multi-gigabyte target (see the Storage bullet above).
- All backends within one scenario/scale run back-to-back in the same script invocation, each against its own freshly-seeded, freshly-created storage (fresh Aerospike set truncation, fresh temp directory, fresh S3 bucket) -- no shared state or warm-cache advantage carries from one backend to the next.

Summary tables

MemoryStore @ scale=50

backend	add	search	search_selective
AerospikeMemoryStore	mean 0.899ms p95 2.613ms 1113 ops/s	mean 5.015ms p95 8.782ms 199 ops/s	mean 3.602ms p95 6.843ms 278 ops/s
FileMemoryStore	mean 0.245ms p95 0.697ms 4076 ops/s	mean 9.401ms p95 18.120ms 106 ops/s	mean 9.151ms p95 14.211ms 109 ops/s

MemoryStore @ scale=250

backend	add	search	search_selective
AerospikeMemoryStore	mean 0.670ms p95 0.849ms 1493 ops/s	mean 6.077ms p95 7.770ms 165 ops/s	mean 2.338ms p95 3.018ms 428 ops/s
FileMemoryStore	mean 0.184ms p95 0.286ms 5430 ops/s	mean 21.463ms p95 25.133ms 47 ops/s	mean 17.173ms p95 20.947ms 58 ops/s

MemoryStore @ scale=1000

backend	add	search	search_selective
AerospikeMemoryStore	mean 0.628ms p95 0.789ms 1591 ops/s	mean 11.667ms p95 13.176ms 86 ops/s	mean 2.318ms p95 2.908ms 431 ops/s
FileMemoryStore	mean 0.159ms p95 0.231ms 6285 ops/s	mean 61.651ms p95 231.722ms 16 ops/s	mean 59.799ms p95 238.391ms 17 ops/s

MemoryStore @ scale=5000

backend	add	search	search_selective
AerospikeMemoryStore	mean 1.272ms p95 1.679ms 786 ops/s	mean 92.302ms p95 102.002ms 11 ops/s	mean 2.503ms p95 2.898ms 400 ops/s
FileMemoryStore	mean 0.456ms p95 0.648ms 2195 ops/s	mean 1437.493ms p95 1672.416ms 1 ops/s	mean 1458.554ms p95 1684.451ms 1 ops/s

SessionRepository @ scale=20

backend	create_message	delete_session	list_messages	read_message
AerospikeSessionManager	mean 0.414ms p95 0.674ms 2413 ops/s	mean 1.256ms p95 2.422ms 796 ops/s	mean 28.576ms p95 41.652ms 35 ops/s	mean 0.431ms p95 0.677ms 2321 ops/s
FileSessionManager	mean 0.348ms p95 0.843ms 2871 ops/s	mean 0.888ms p95 0.956ms 1126 ops/s	mean 34.185ms p95 46.784ms 29 ops/s	mean 0.417ms p95 0.874ms 2400 ops/s
S3SessionManager	mean 6.858ms p95 9.039ms 146 ops/s	mean 34.232ms p95 48.475ms 29 ops/s	mean 940.153ms p95 1098.506ms 1 ops/s	mean 5.766ms p95 6.601ms 173 ops/s

SessionRepository @ scale=100

backend	create_message	list_messages	read_message
AerospikeSessionManager	mean 0.390ms p95 0.488ms 2567 ops/s	mean 38.154ms p95 43.373ms 26 ops/s	mean 0.385ms p95 0.510ms 2595 ops/s
FileSessionManager	mean 0.224ms p95 0.317ms 4455 ops/s	mean 44.540ms p95 50.254ms 22 ops/s	mean 0.269ms p95 0.395ms 3719 ops/s
S3SessionManager	mean 6.801ms p95 7.964ms 147 ops/s	mean 1556.385ms p95 1804.325ms 1 ops/s	mean 5.966ms p95 6.945ms 168 ops/s

SessionRepository @ scale=500

backend	create_message	list_messages	read_message
AerospikeSessionManager	mean 0.434ms p95 0.600ms 2304 ops/s	mean 122.299ms p95 143.559ms 8 ops/s	mean 0.411ms p95 0.614ms 2431 ops/s
FileSessionManager	mean 0.235ms p95 0.335ms 4262 ops/s	mean 126.752ms p95 145.246ms 8 ops/s	mean 0.258ms p95 0.365ms 3872 ops/s
S3SessionManager	mean 6.801ms p95 16.214ms 147 ops/s	mean 4052.012ms p95 4340.192ms 0 ops/s	mean 6.441ms p95 8.124ms 155 ops/s

SessionRepository @ scale=2000

backend	create_message	list_messages	read_message
AerospikeSessionManager	mean 0.480ms p95 0.658ms 2082 ops/s	mean 438.758ms p95 607.964ms 2 ops/s	mean 0.450ms p95 0.639ms 2222 ops/s
FileSessionManager	mean 0.313ms p95 0.415ms 3195 ops/s	mean 478.056ms p95 548.866ms 2 ops/s	mean 0.295ms p95 0.419ms 3395 ops/s

backend	create_message	list_messages	read_message
S3SessionManager	mean 6.583ms p95 7.693ms 152 ops/s	mean 14517.594ms p95 15965.558ms 0 ops/s	mean 6.097ms p95 7.144ms 164 ops/s

Storage @ scale=100

backend	write	delete	list	read	search
AerospikeStorage	mean 0.698ms p95 1.413ms 1432 ops/s	mean 0.482ms p95 0.658ms 2075 ops/s	mean 24.237ms p95 38.146ms 41 ops/s	mean 0.311ms p95 0.435ms 3219 ops/s	mean 137.794ms p95 161.045ms 7 ops/s
InMemoryStorage	mean 0.008ms p95 0.021ms 128463 ops/s	mean 0.007ms p95 0.019ms 133499 ops/s	mean 0.046ms p95 0.099ms 21605 ops/s	mean 0.007ms p95 0.015ms 153588 ops/s	mean 21.222ms p95 24.212ms 47 ops/s
LocalFileStorage	mean 0.118ms p95 0.182ms 8459 ops/s	mean 0.031ms p95 0.057ms 32664 ops/s	mean 1.643ms p95 2.185ms 609 ops/s	mean 0.026ms p95 0.032ms 38939 ops/s	mean 35.856ms p95 40.290ms 28 ops/s
S3Storage	mean 10.670ms p95 20.082ms 94 ops/s	mean 7.009ms p95 10.137ms 143 ops/s	mean 51.956ms p95 66.103ms 19 ops/s	mean 9.019ms p95 12.685ms 111 ops/s	mean 1824.754ms p95 2133.117ms 1 ops/s

Storage @ scale=500

backend	write	delete	list	read	search
AerospikeStorage	mean 0.510ms p95 0.628ms 1962 ops/s	mean 0.477ms p95 0.709ms 2095 ops/s	mean 23.833ms p95 35.657ms 42 ops/s	mean 0.356ms p95 0.729ms 2806 ops/s	mean 298.955ms p95 326.686ms 3 ops/s
InMemoryStorage	mean 0.008ms p95 0.018ms 127099 ops/s	mean 0.007ms p95 0.018ms 134242 ops/s	mean 0.111ms p95 0.185ms 9049 ops/s	mean 0.007ms p95 0.017ms 146259 ops/s	mean 53.720ms p95 59.344ms 19 ops/s
LocalFileStorage	mean 0.104ms p95 0.179ms 9649 ops/s	mean 0.025ms p95 0.035ms 40811 ops/s	mean 5.214ms p95 11.300ms 192 ops/s	mean 0.027ms p95 0.036ms 37512 ops/s	mean 82.939ms p95 164.441ms 12 ops/s

backend	write	delete	list	read	search
S3Storage	mean 7.997ms p95 16.650ms 125 ops/s	mean 5.438ms p95 6.230ms 184 ops/s	mean 185.132ms p95 295.079ms 5 ops/s	mean 6.518ms p95 7.740ms 153 ops/s	mean 4442.390ms p95 4620.981ms 0 ops/s

Storage @ scale=2000

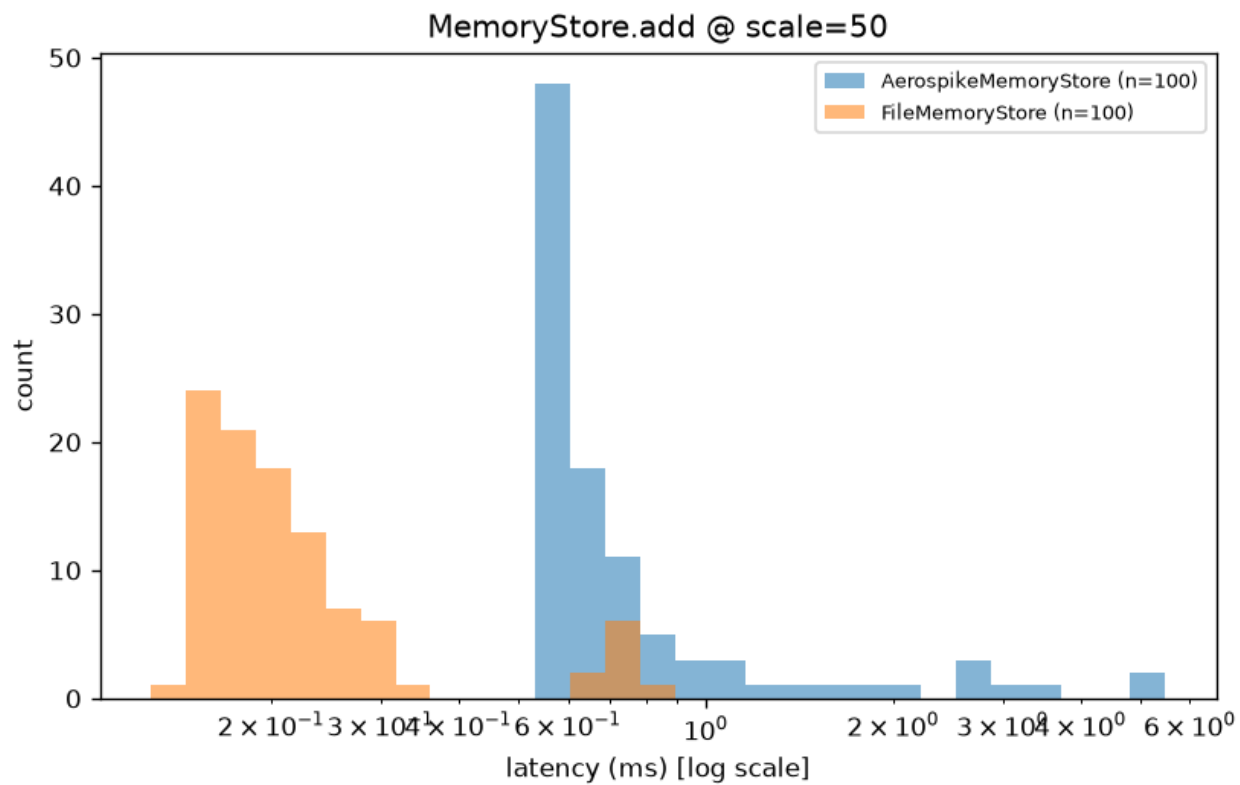
backend	write	delete	list	read
AerospikeStorage	mean 0.609ms p95 0.814ms 1642 ops/s	mean 0.463ms p95 0.583ms 2161 ops/s	mean 26.950ms p95 28.011ms 37 ops/s	mean 0.357ms p95 0.583ms 2800 ops/s
InMemoryStorage	mean 0.007ms p95 0.016ms 135746 ops/s	mean 0.003ms p95 0.006ms 328404 ops/s	mean 0.340ms p95 0.440ms 2939 ops/s	mean 0.006ms p95 0.019ms 172692 ops/s
LocalFileStorage	mean 0.106ms p95 0.173ms 9451 ops/s	mean 0.026ms p95 0.035ms 38261 ops/s	mean 14.441ms p95 19.545ms 69 ops/s	mean 0.030ms p95 0.038ms 33562 ops/s
S3Storage	mean 6.798ms p95 9.423ms 147 ops/s	mean 5.398ms p95 6.228ms 185 ops/s	mean 750.907ms p95 922.180ms 1 ops/s	mean 6.011ms p95 6.733ms 166 ops/s

Storage @ scale=20480

backend	write	delete	list	read
AerospikeStorage	mean 1.323ms p95 1.631ms 756 ops/s	mean 1.294ms p95 1.969ms 773 ops/s	mean 80818.735ms p95 80820.254ms 0 ops/s	mean 2.746ms p95 4.586ms 364 ops/s
LocalFileStorage	mean 0.395ms p95 0.594ms 2532 ops/s	mean 0.075ms p95 0.247ms 13373 ops/s	mean 139.007ms p95 148.670ms 7 ops/s	mean 7.988ms p95 8.453ms 125 ops/s

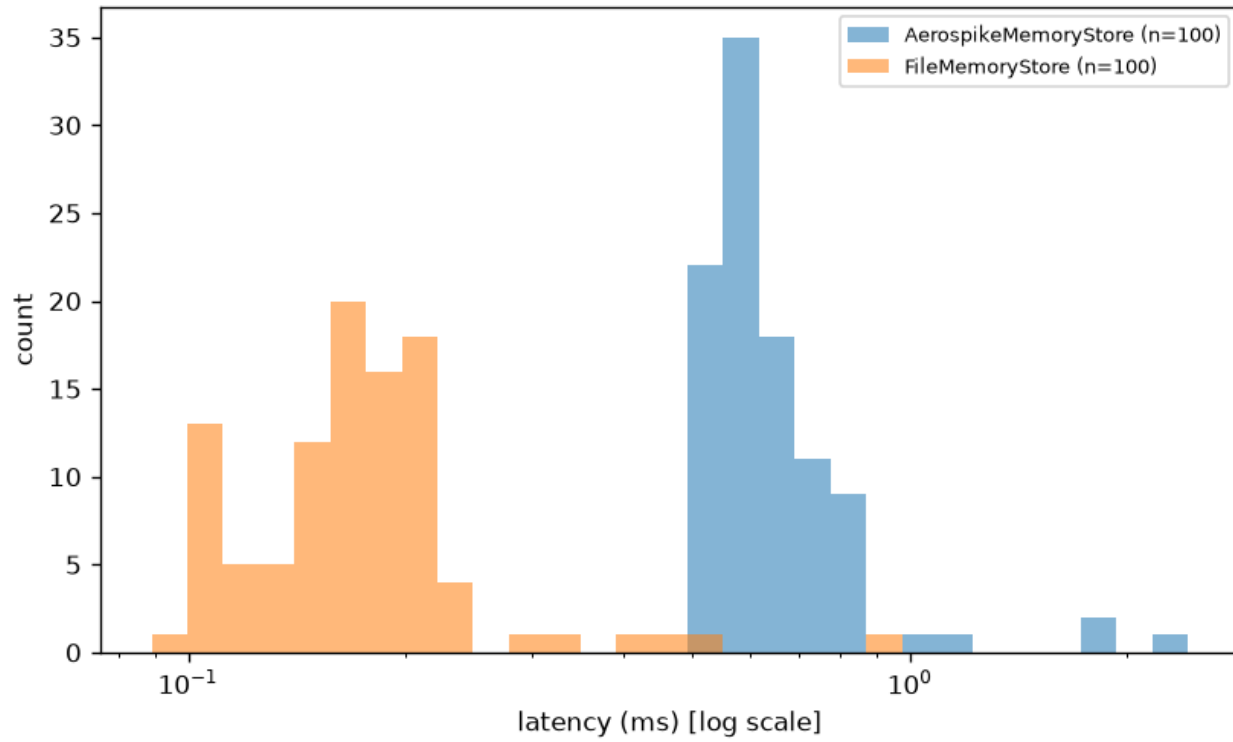
Histograms

MemoryStore.add @ scale=50



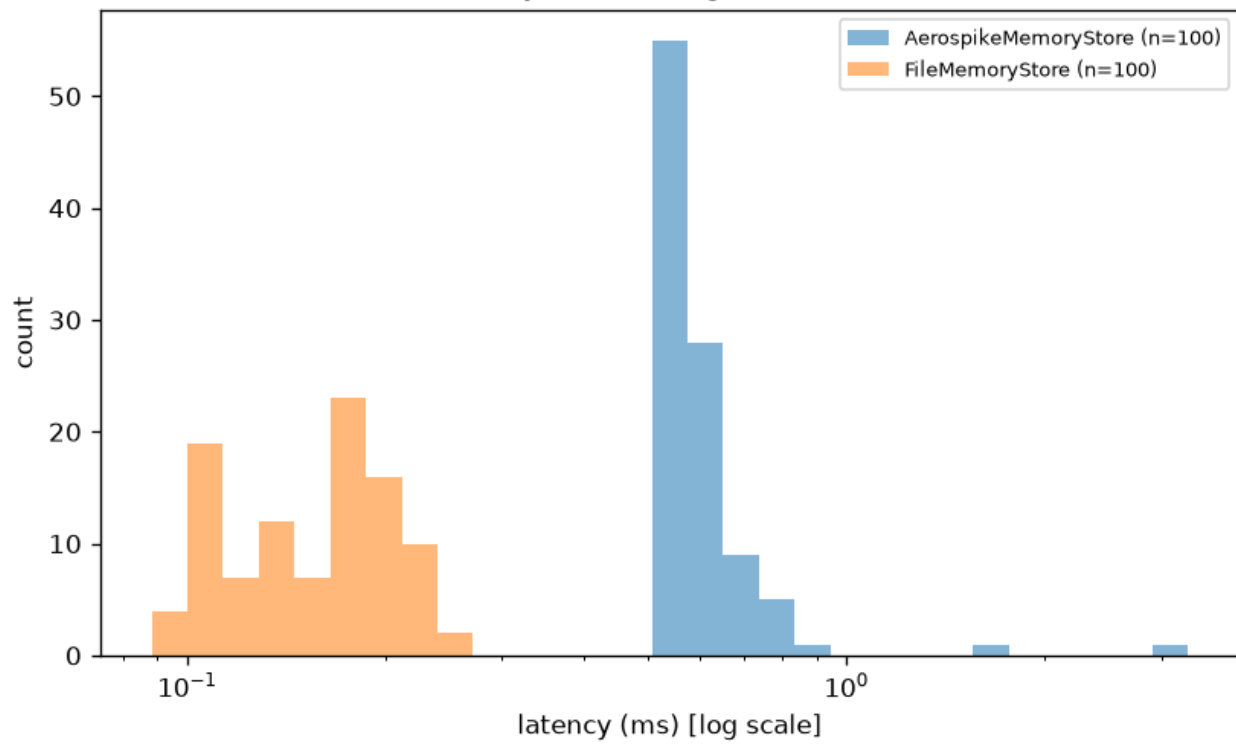
MemoryStore.add @ scale=250

MemoryStore.add @ scale=250



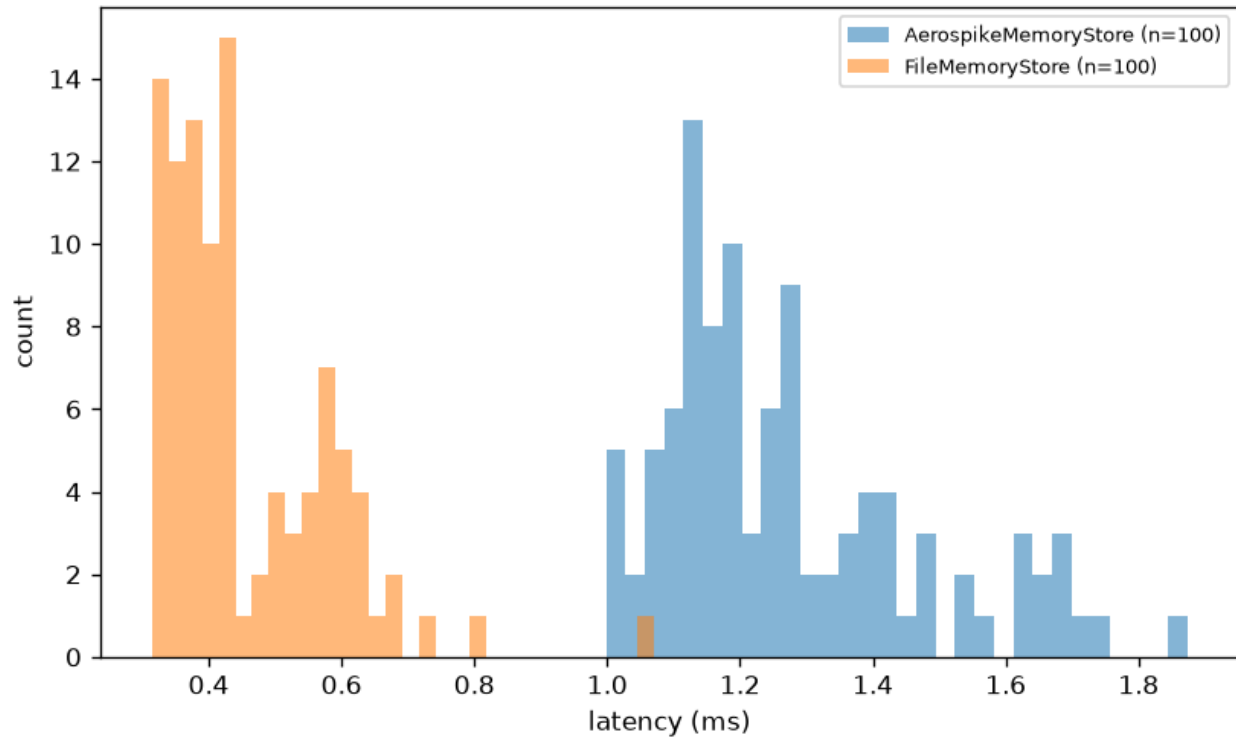
MemoryStore.add @ scale=1000

MemoryStore.add @ scale=1000



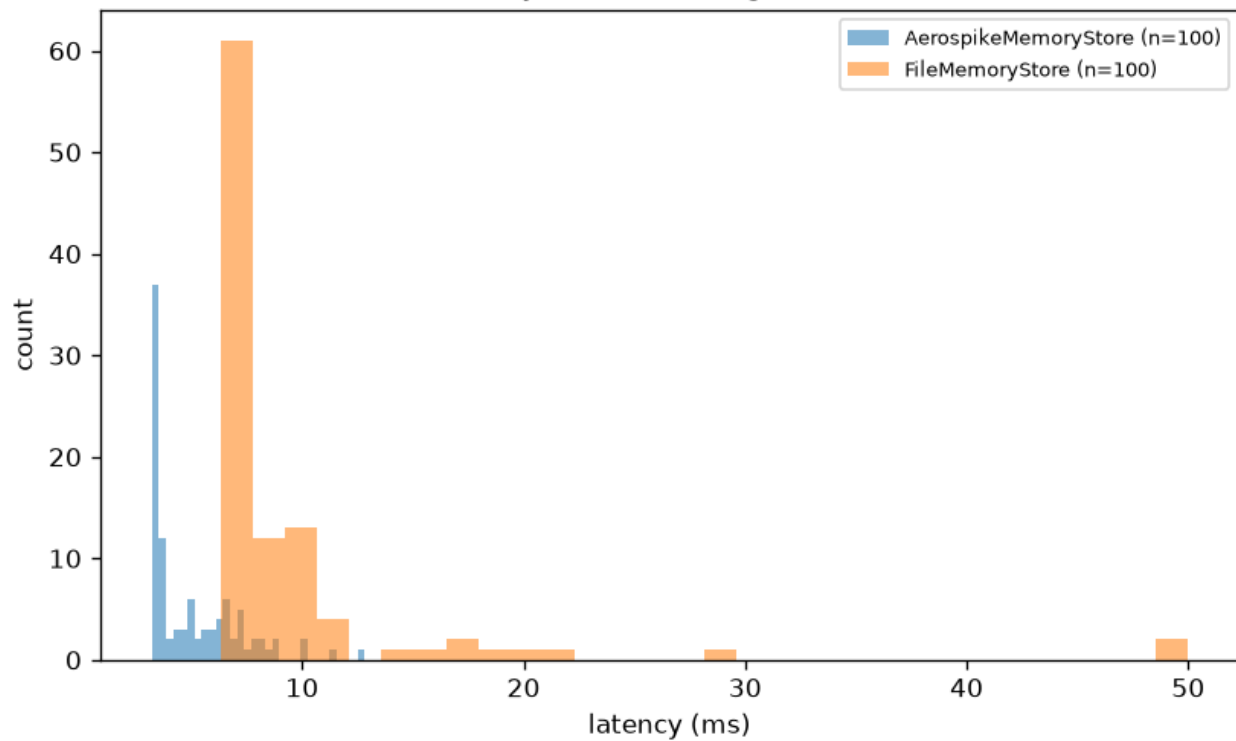
MemoryStore.add @ scale=5000

MemoryStore.add @ scale=5000



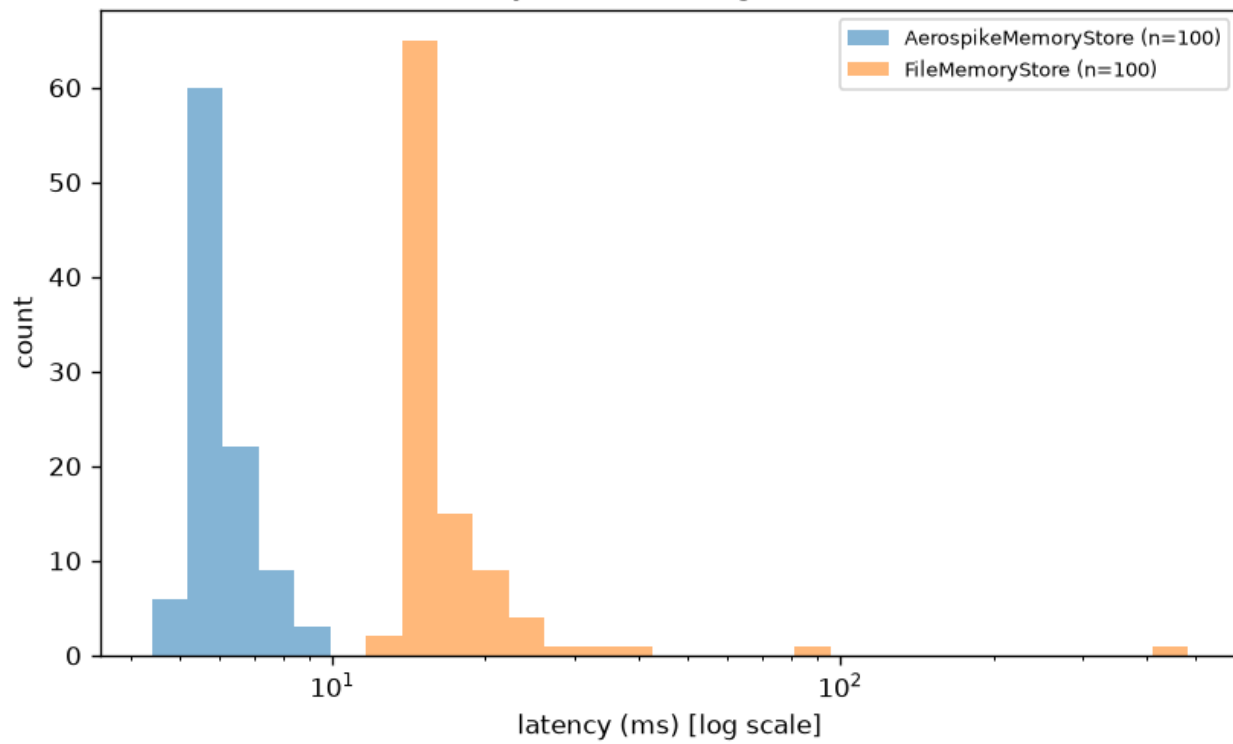
MemoryStore.search @ scale=50

MemoryStore.search @ scale=50



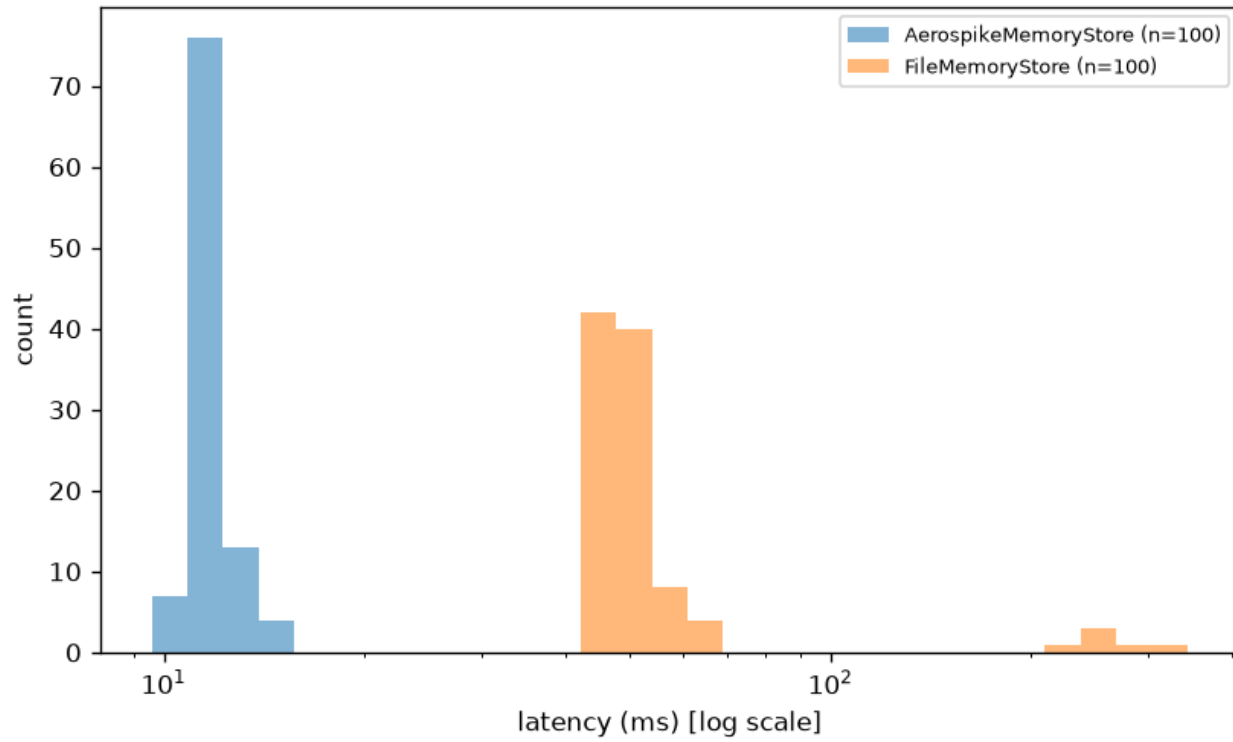
MemoryStore.search @ scale=250

MemoryStore.search @ scale=250



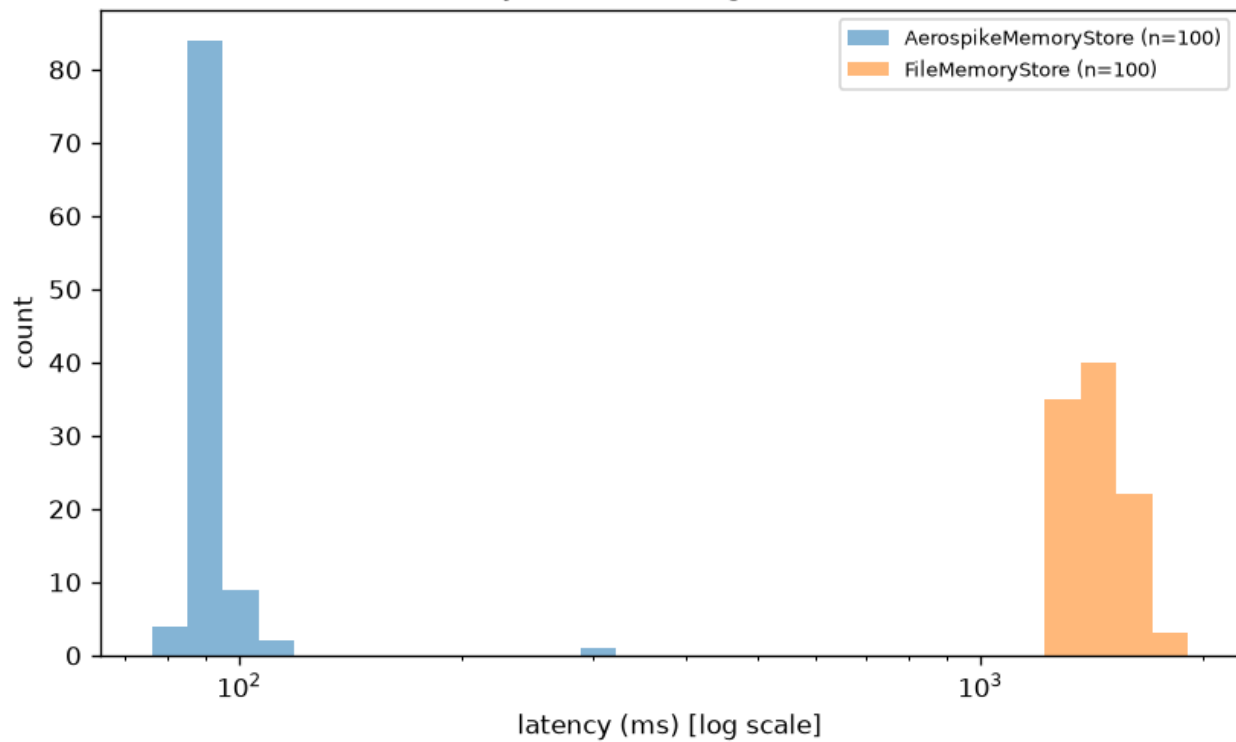
MemoryStore.search @ scale=1000

MemoryStore.search @ scale=1000



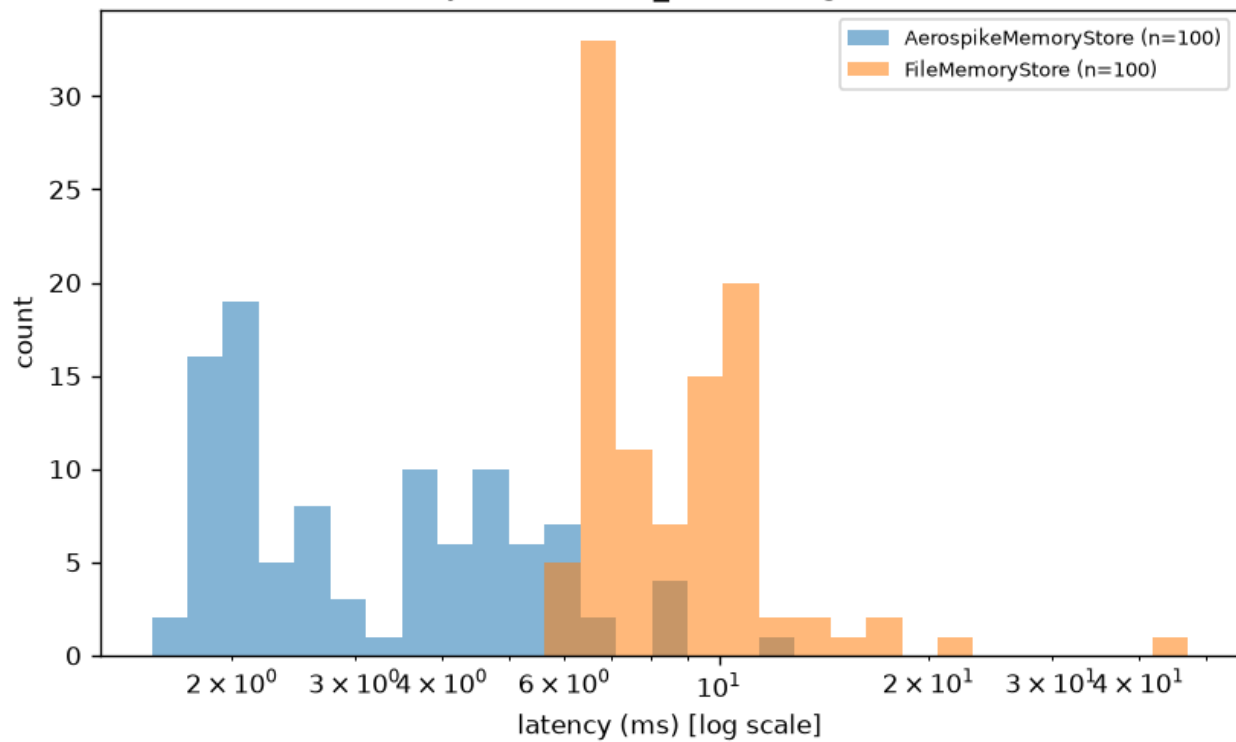
MemoryStore.search @ scale=5000

MemoryStore.search @ scale=5000



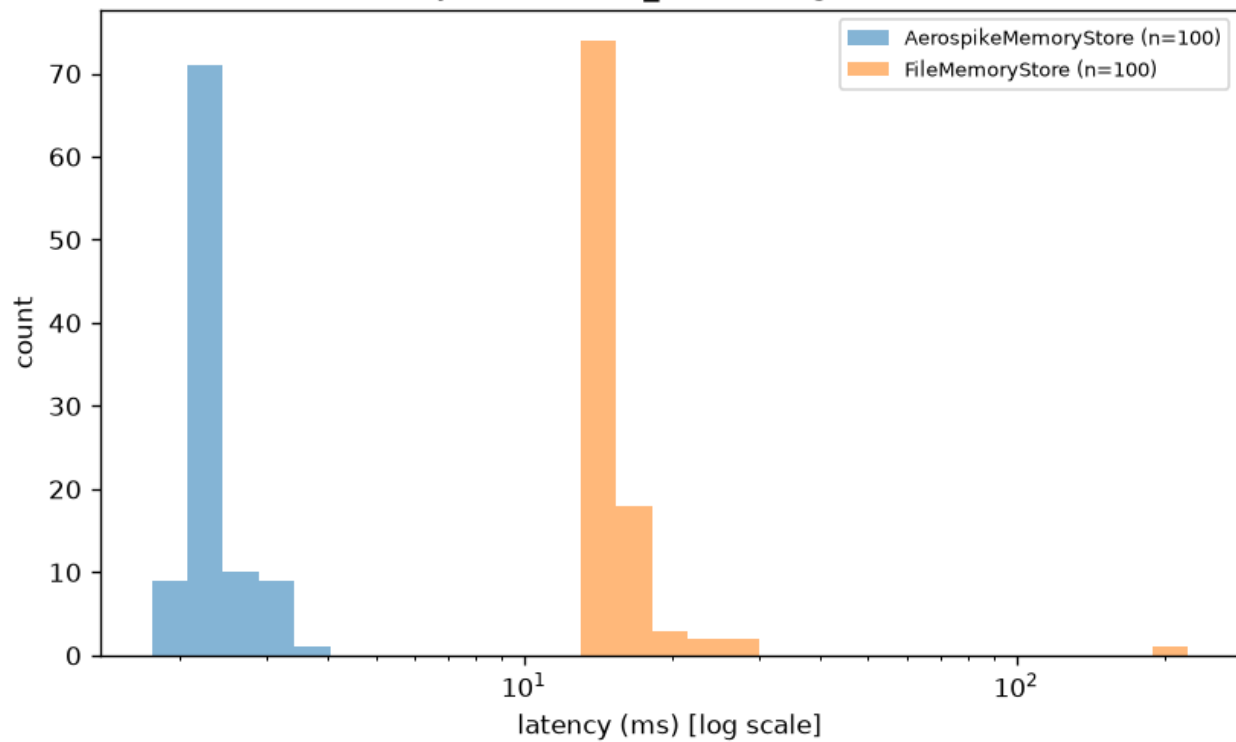
MemoryStore.search_selective @ scale=50

MemoryStore.search_selective @ scale=50

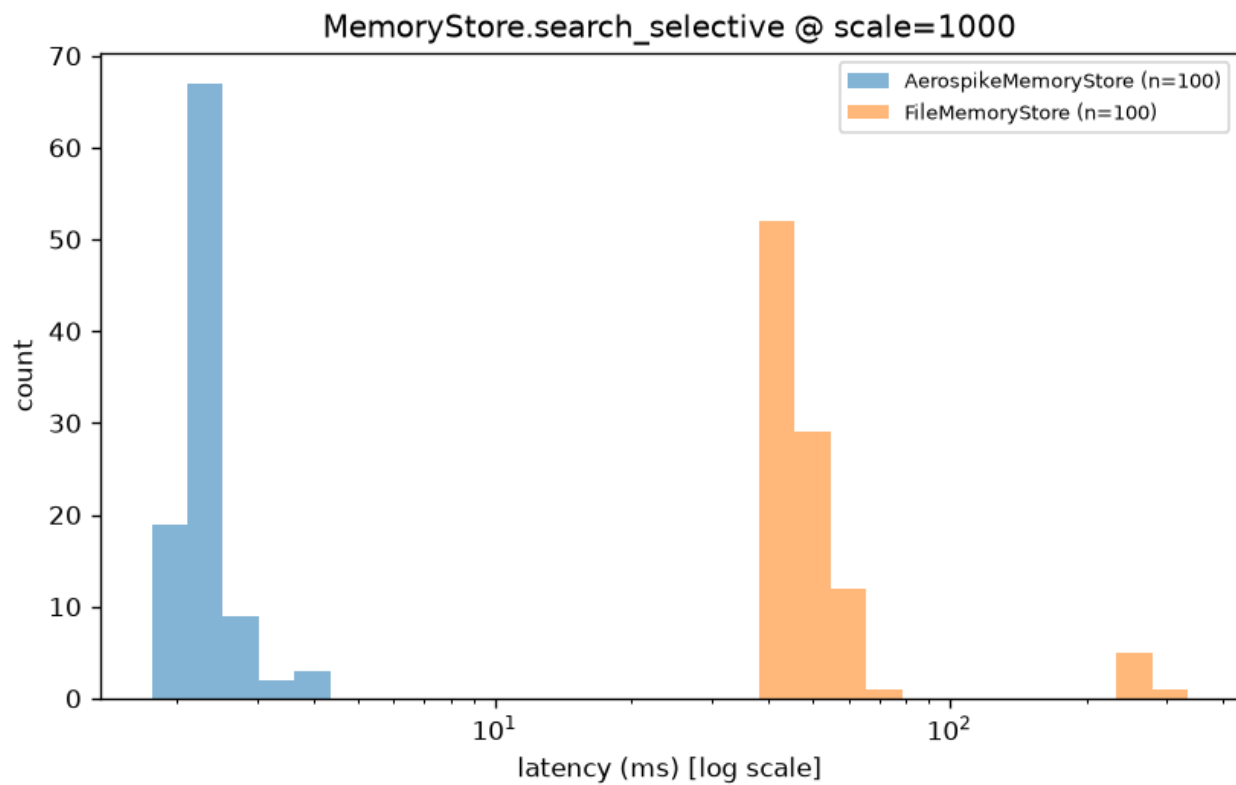


MemoryStore.search_selective @ scale=250

MemoryStore.search_selective @ scale=250

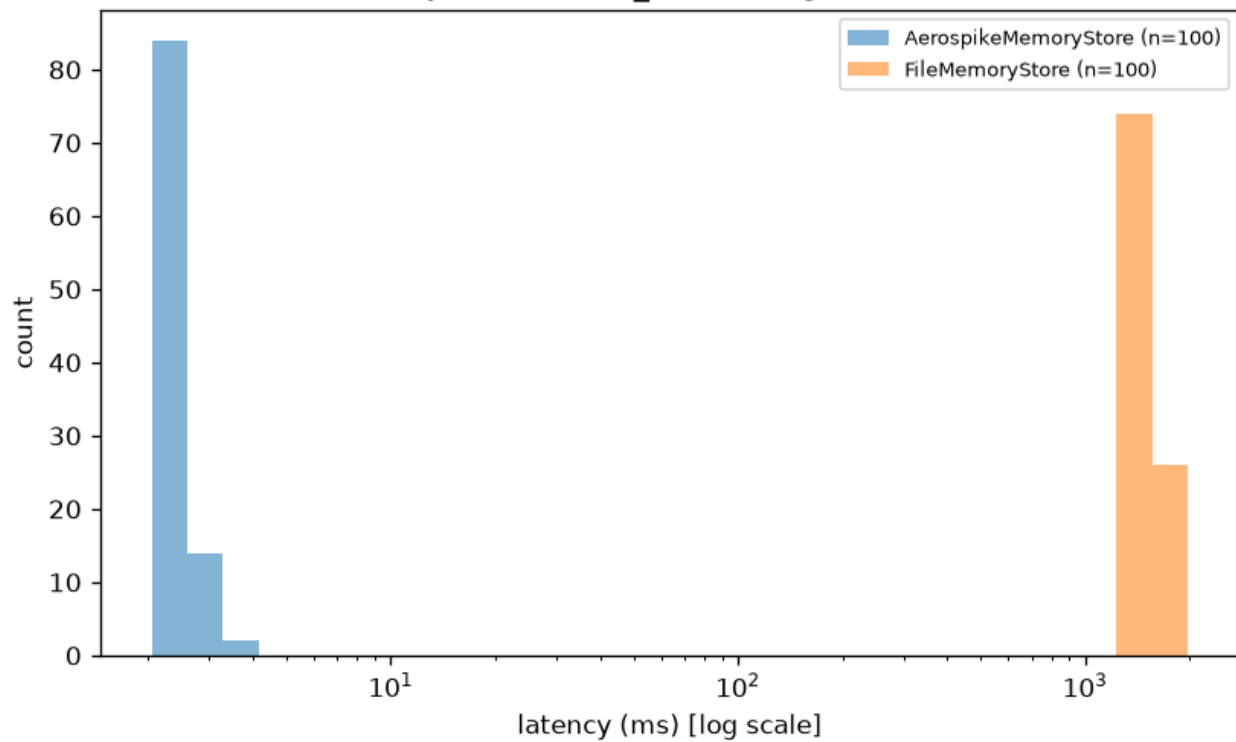


MemoryStore.search_selective @ scale=1000



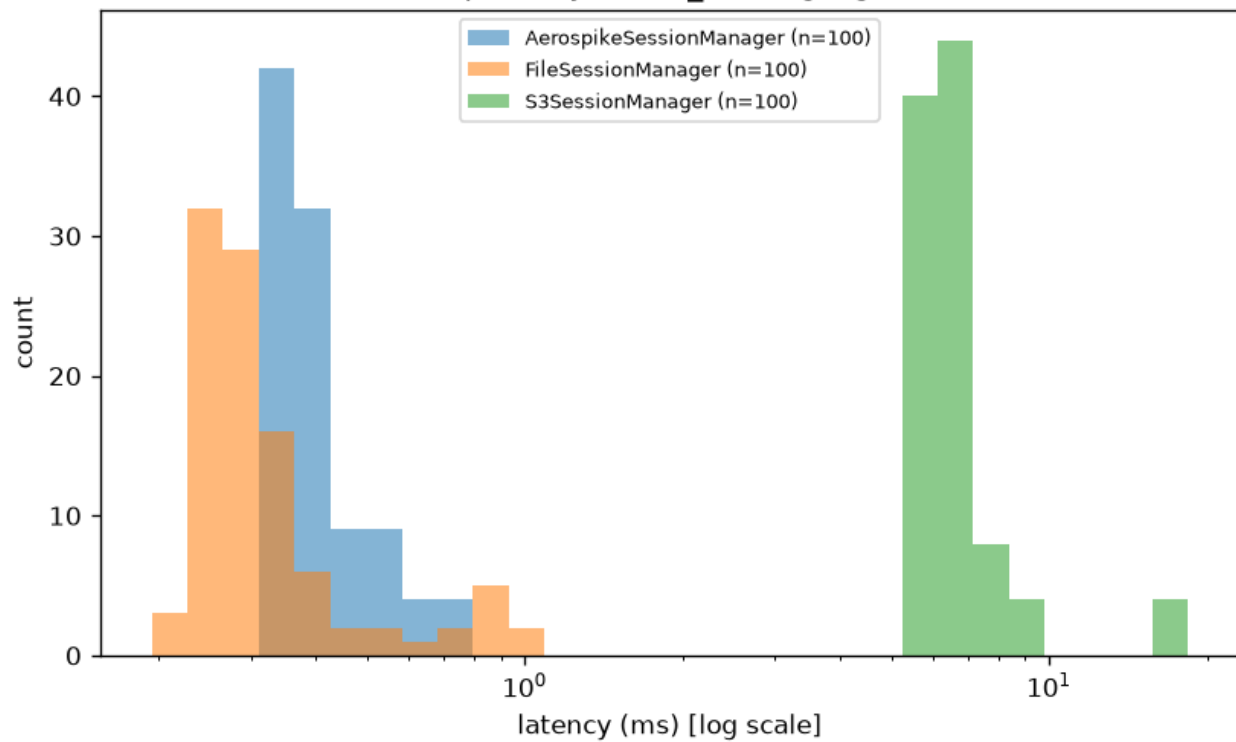
MemoryStore.search_selective @ scale=5000

MemoryStore.search_selective @ scale=5000



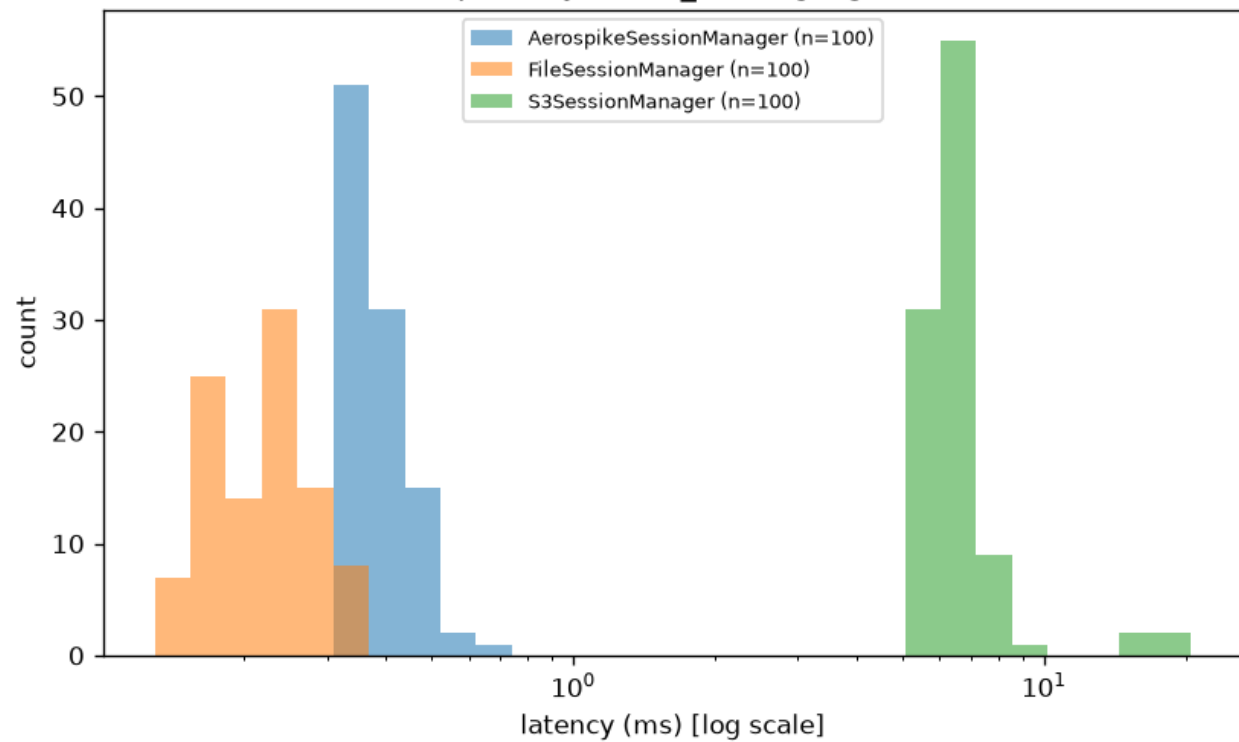
SessionRepository.create_message @ scale=20

SessionRepository.create_message @ scale=20



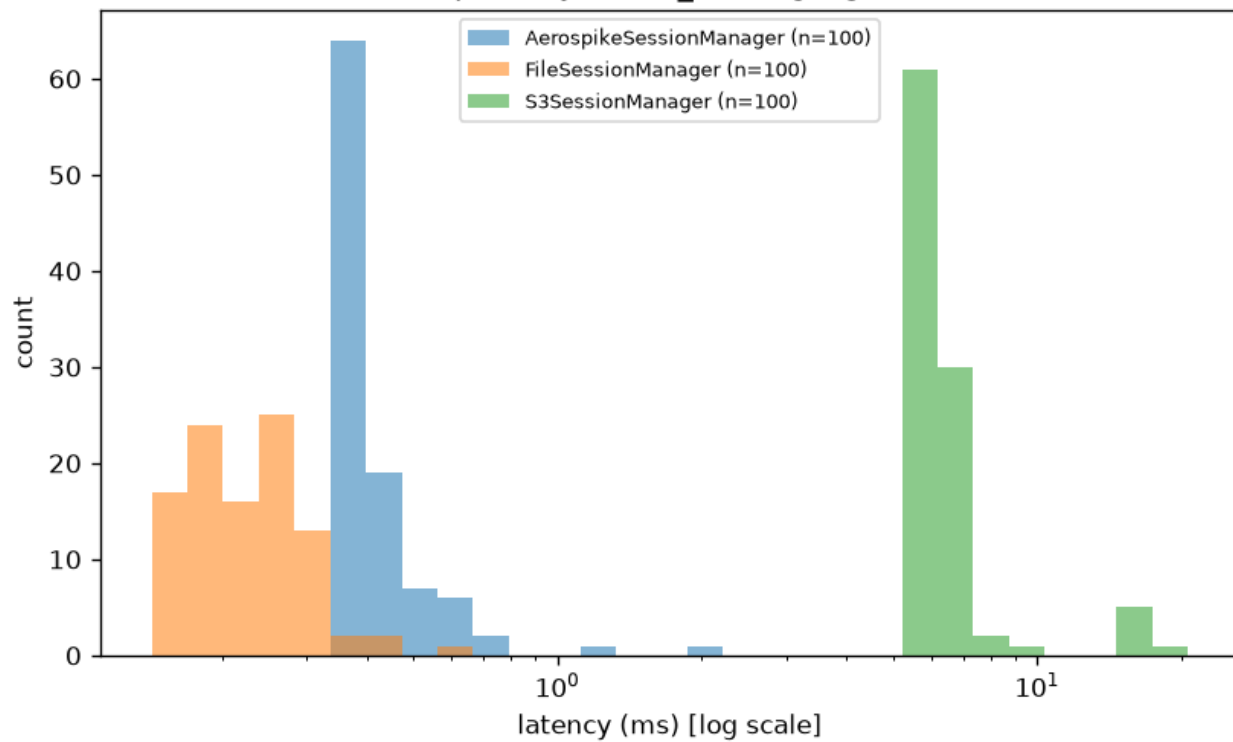
SessionRepository.create_message @ scale=100

SessionRepository.create_message @ scale=100



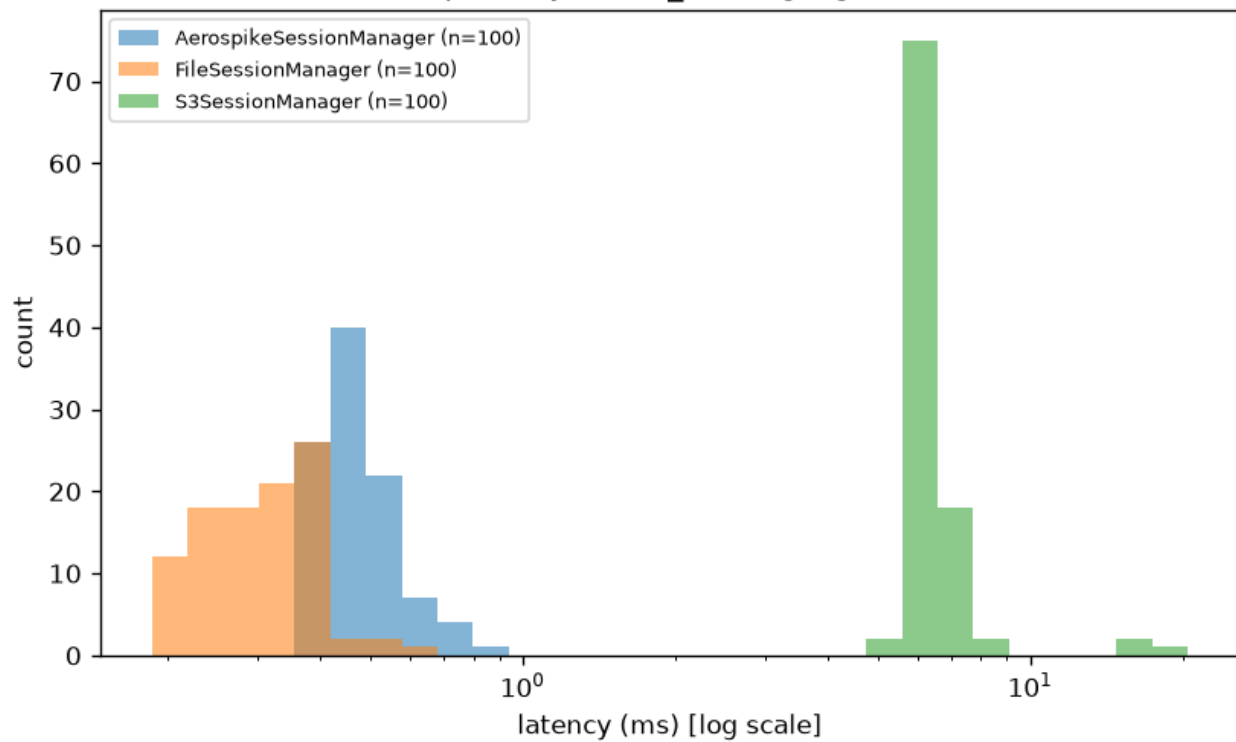
SessionRepository.create_message @ scale=500

SessionRepository.create_message @ scale=500



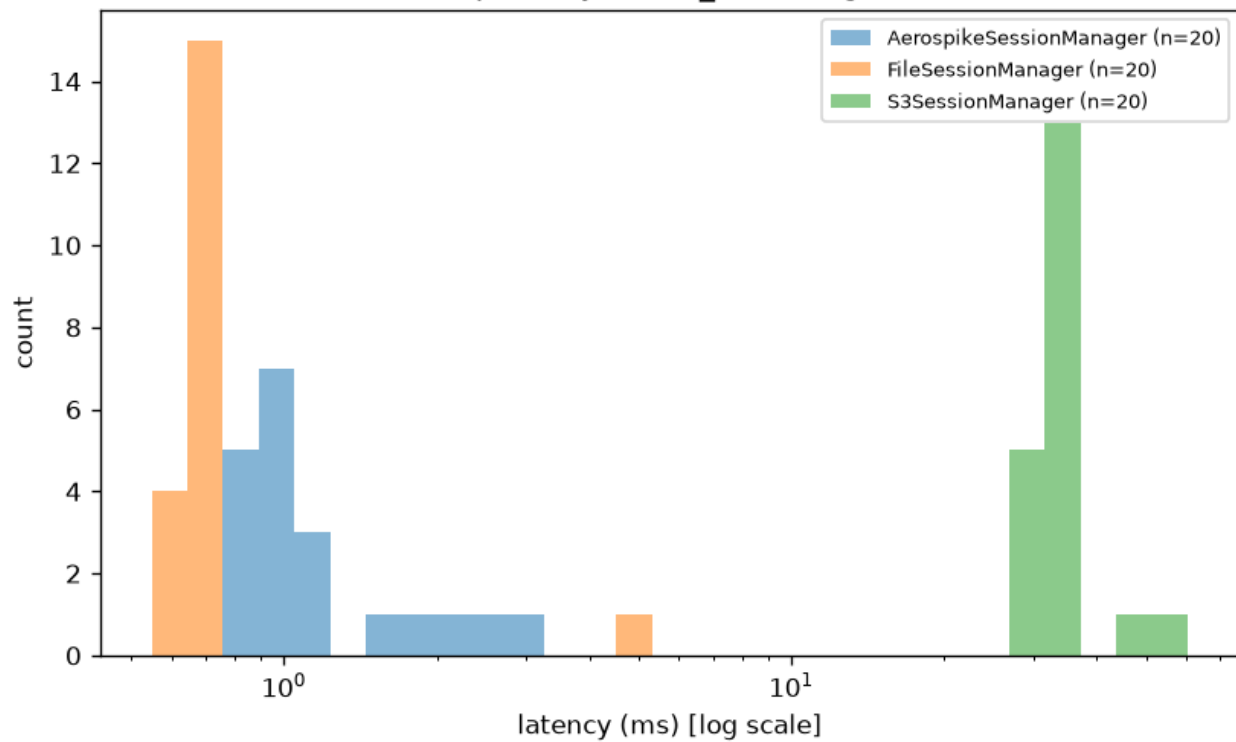
SessionRepository.create_message @ scale=2000

SessionRepository.create_message @ scale=2000



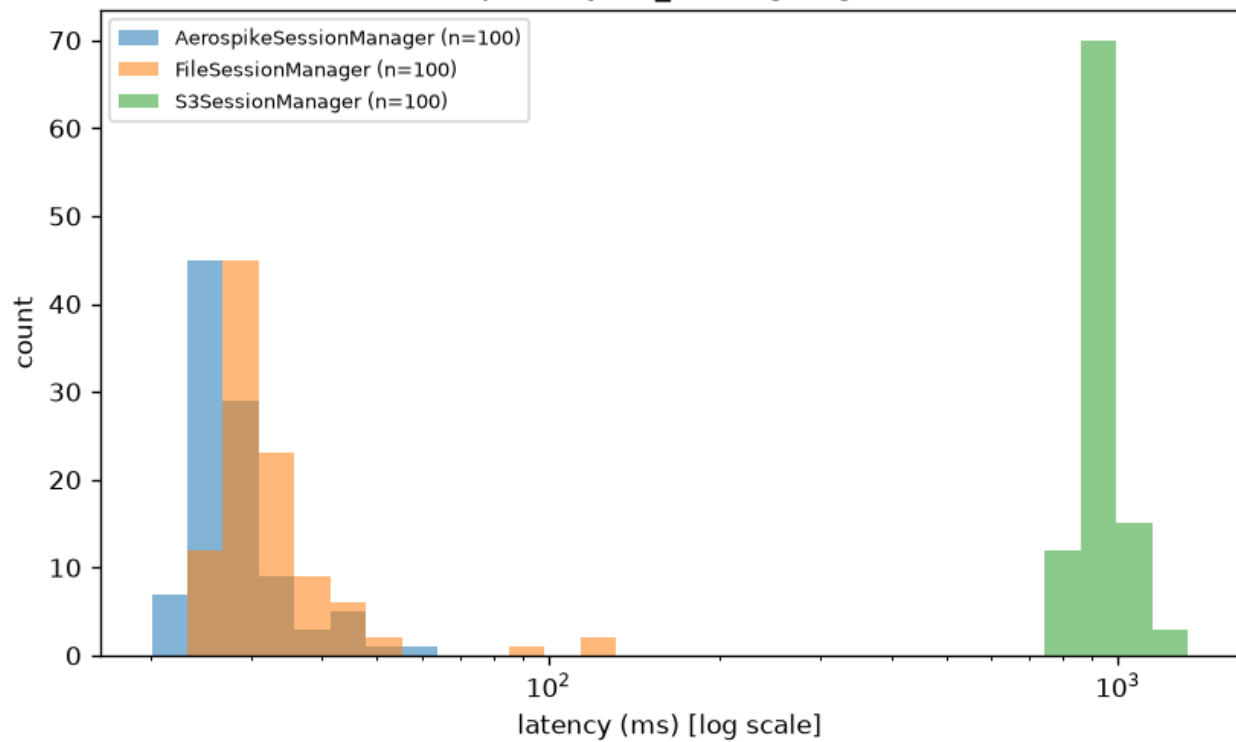
SessionRepository.delete_session @ scale=20

SessionRepository.delete_session @ scale=20



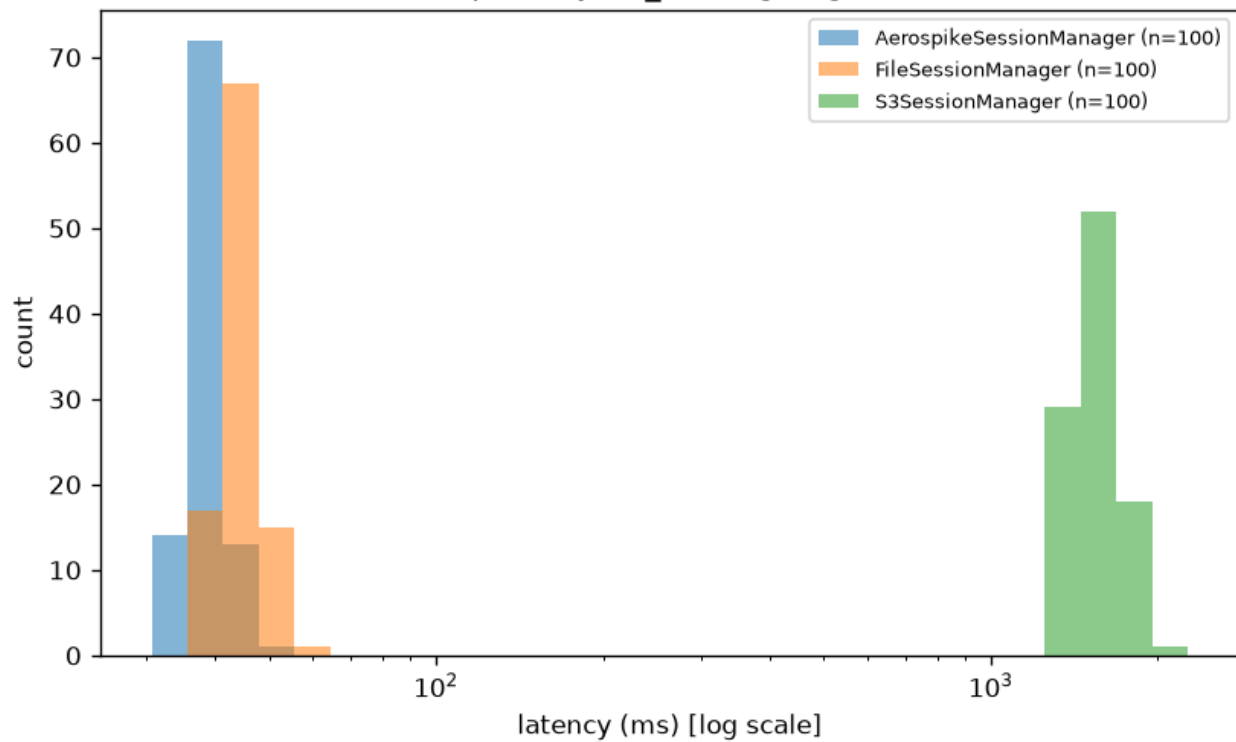
SessionRepository.list_messages @ scale=20

SessionRepository.list_messages @ scale=20



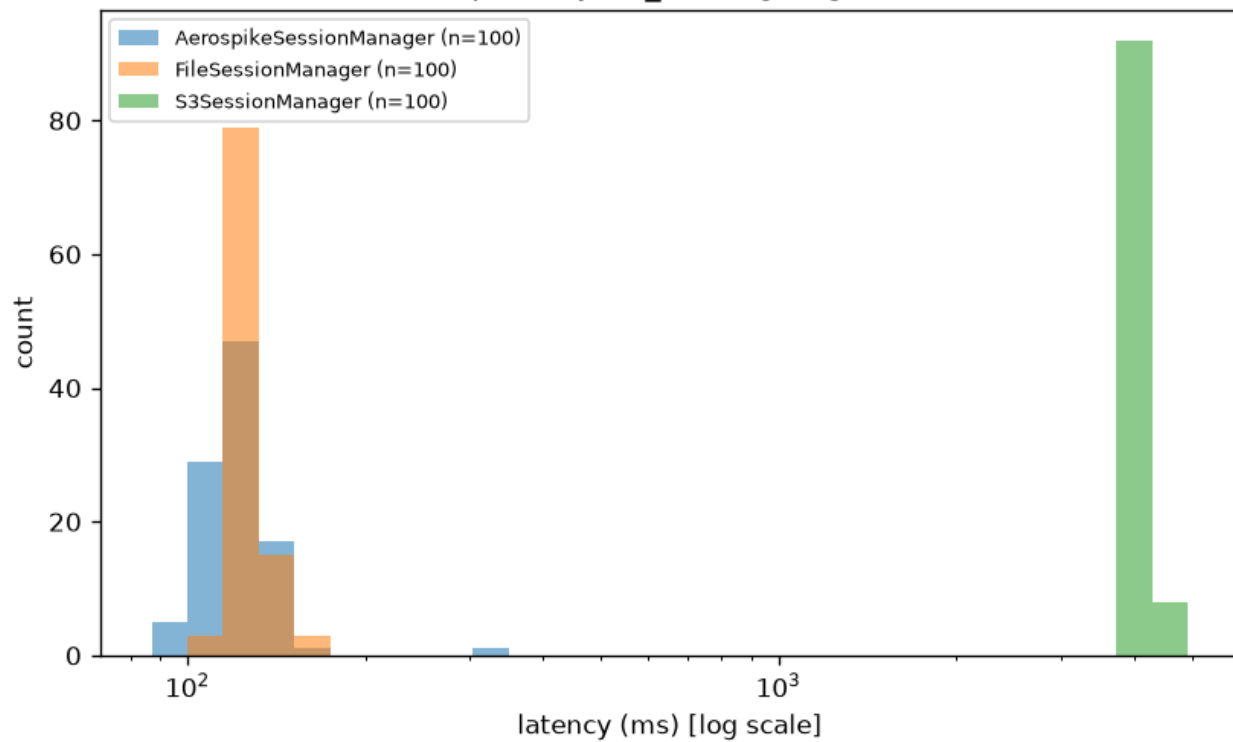
SessionRepository.list_messages @ scale=100

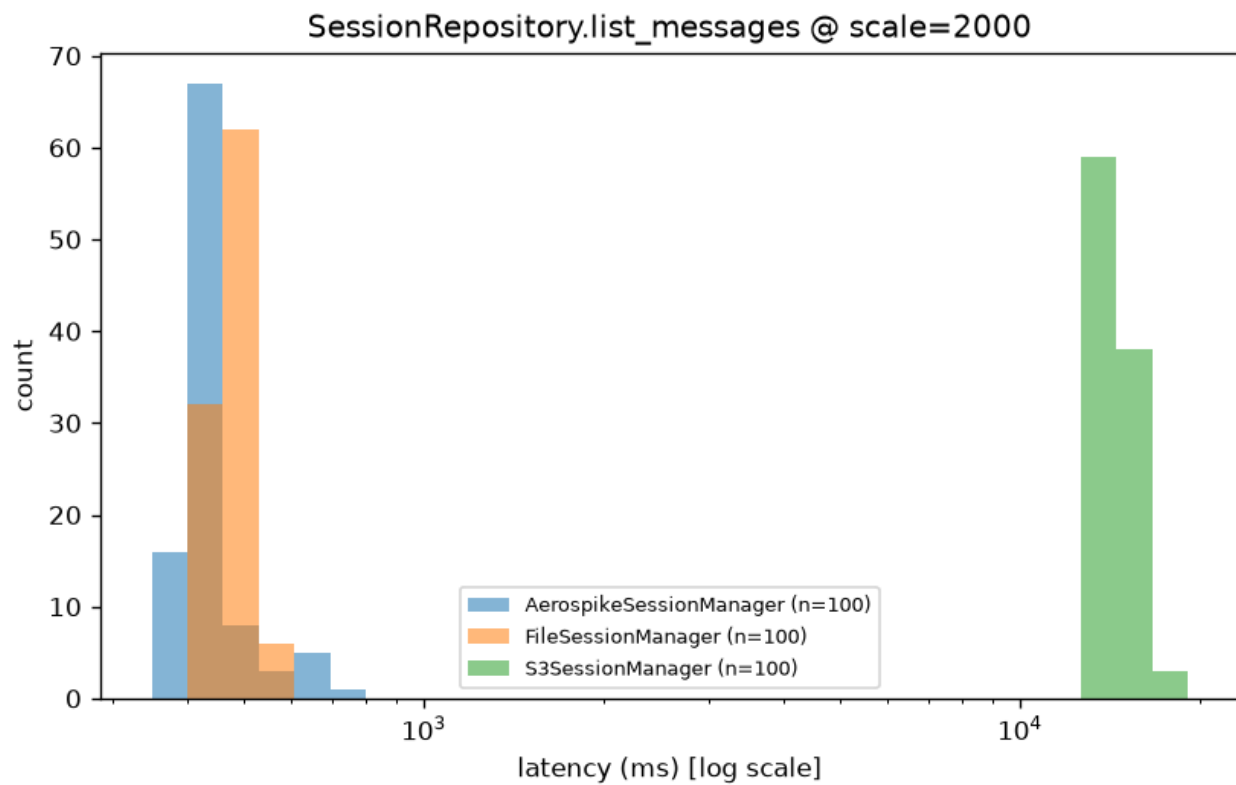
SessionRepository.list_messages @ scale=100



SessionRepository.list_messages @ scale=500

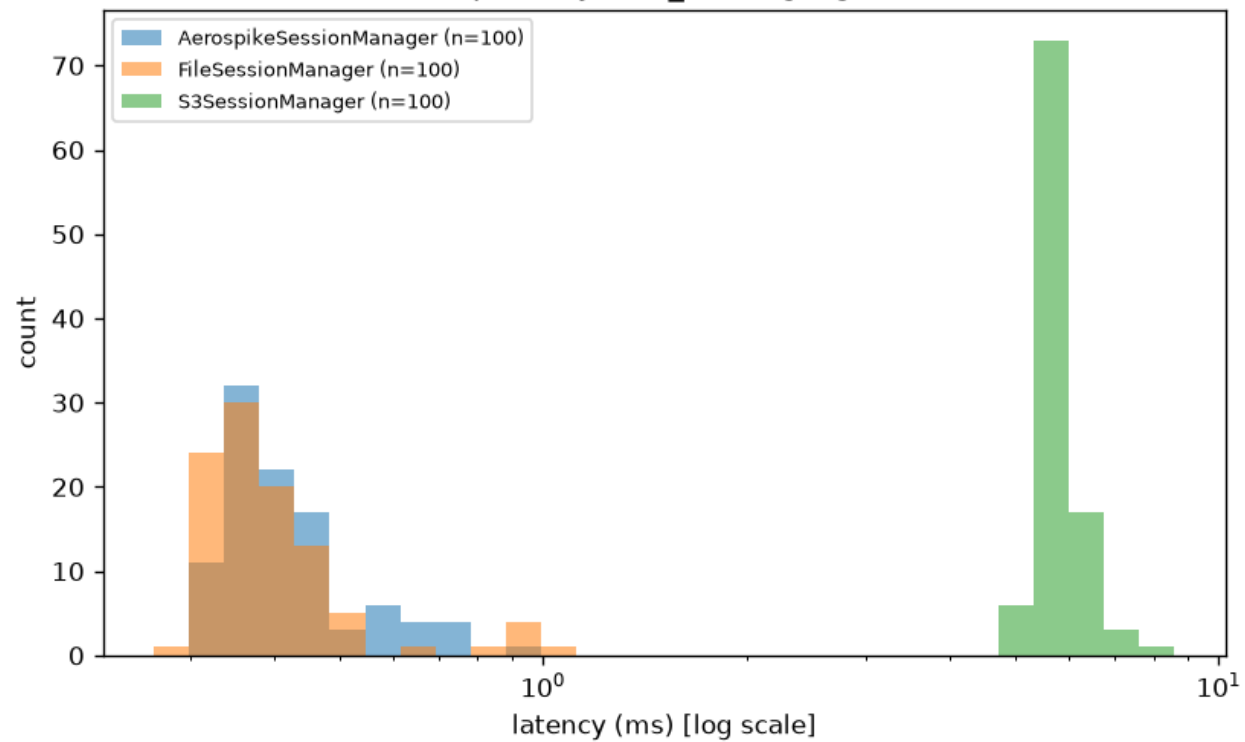
SessionRepository.list_messages @ scale=500



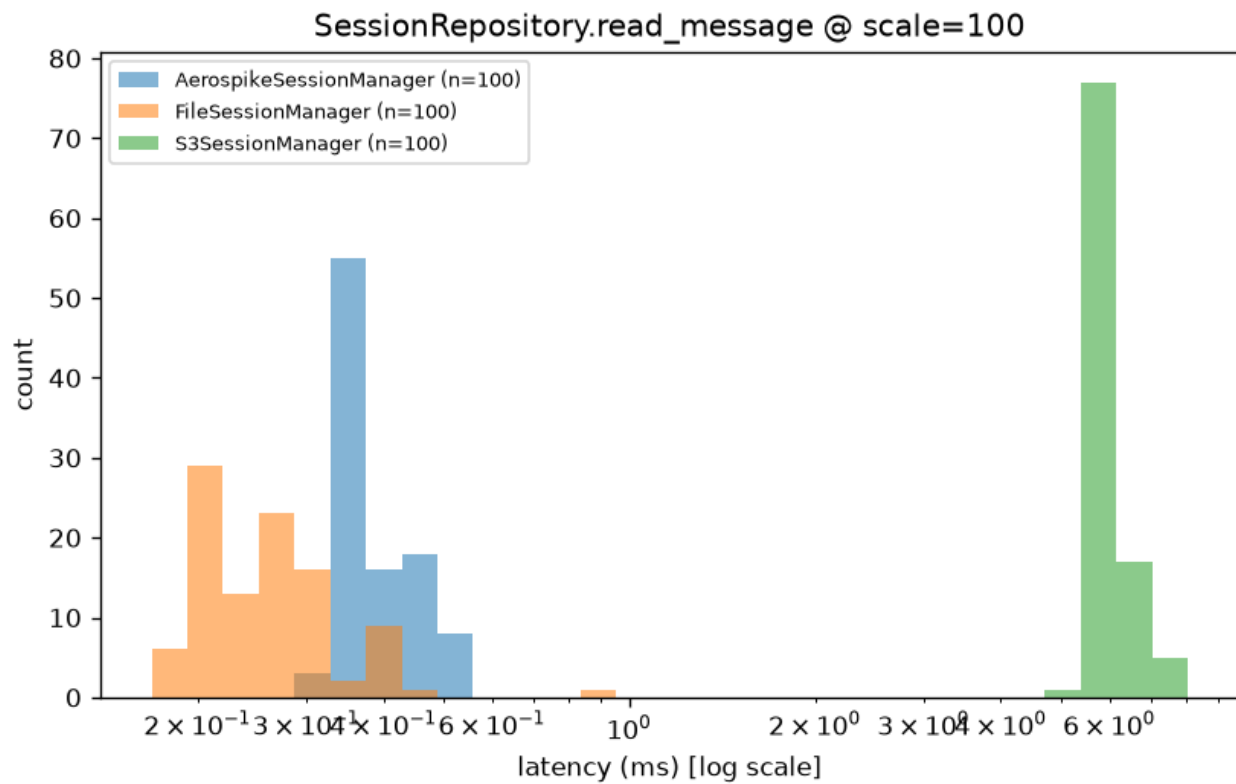


SessionRepository.read_message @ scale=20

SessionRepository.read_message @ scale=20

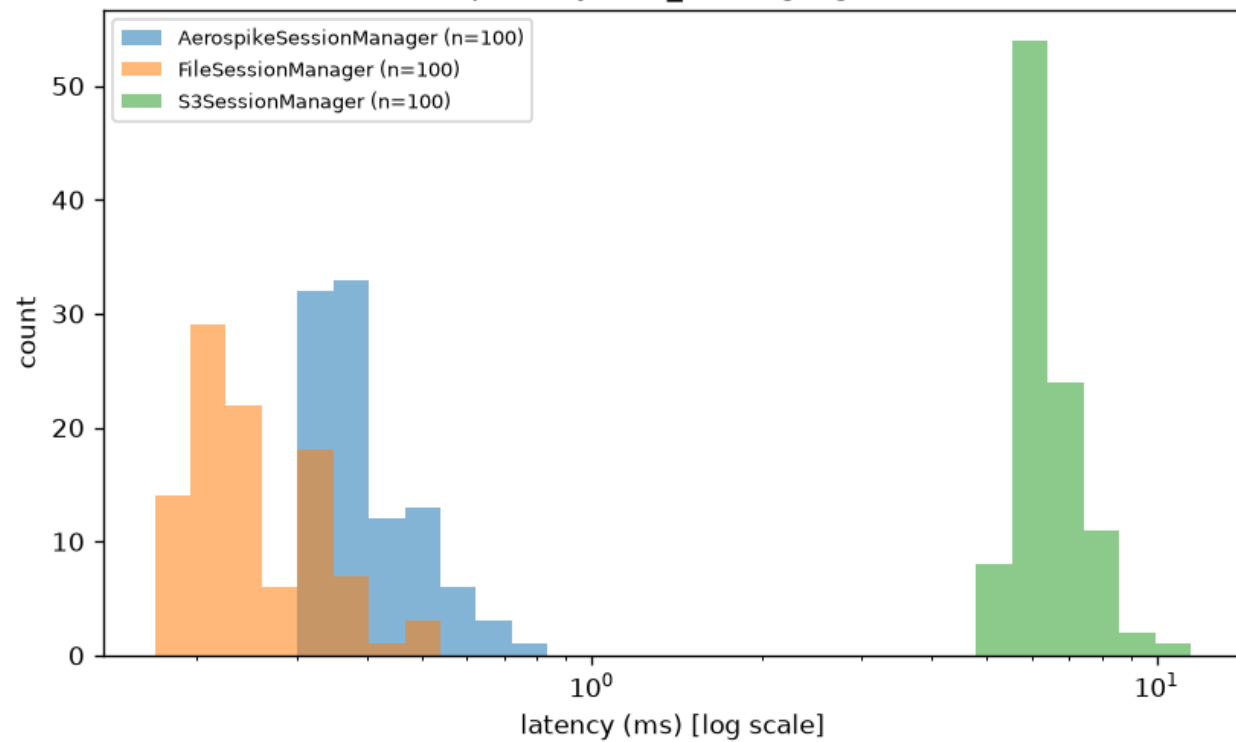


SessionRepository.read_message @ scale=100

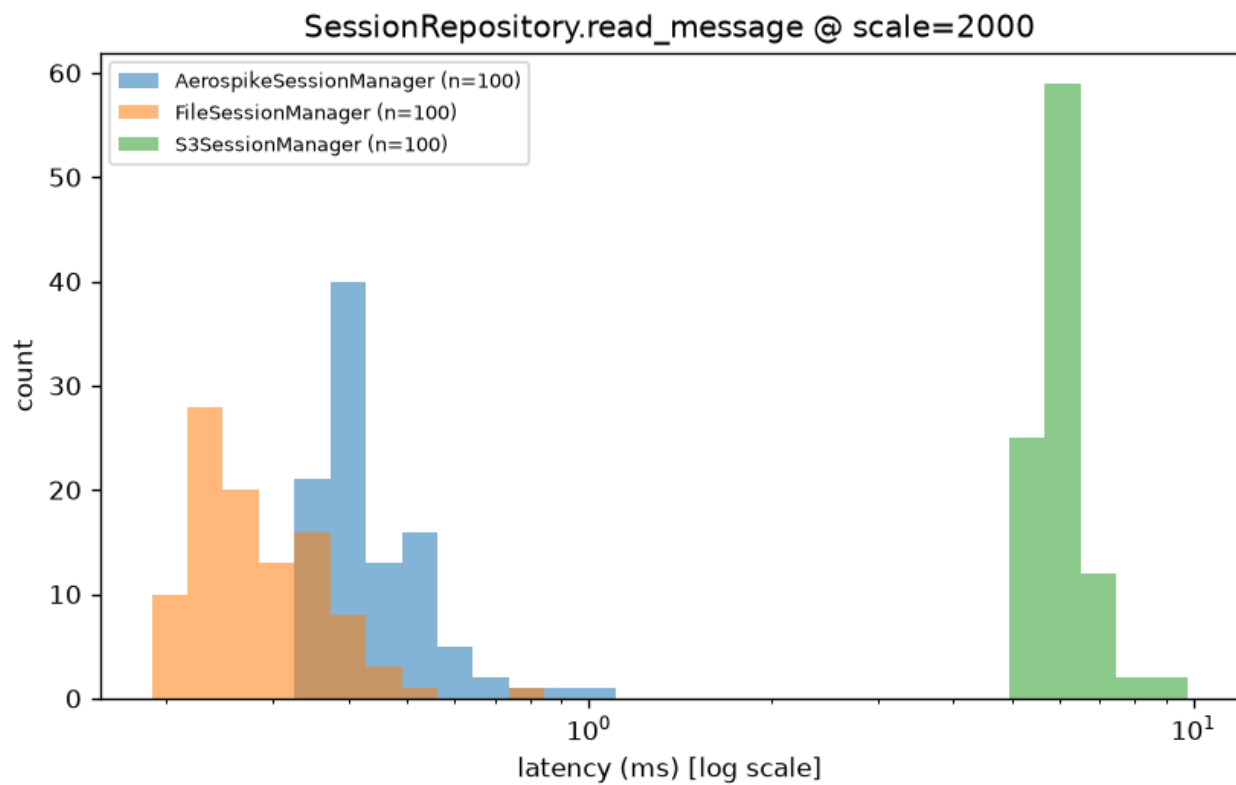


SessionRepository.read_message @ scale=500

SessionRepository.read_message @ scale=500

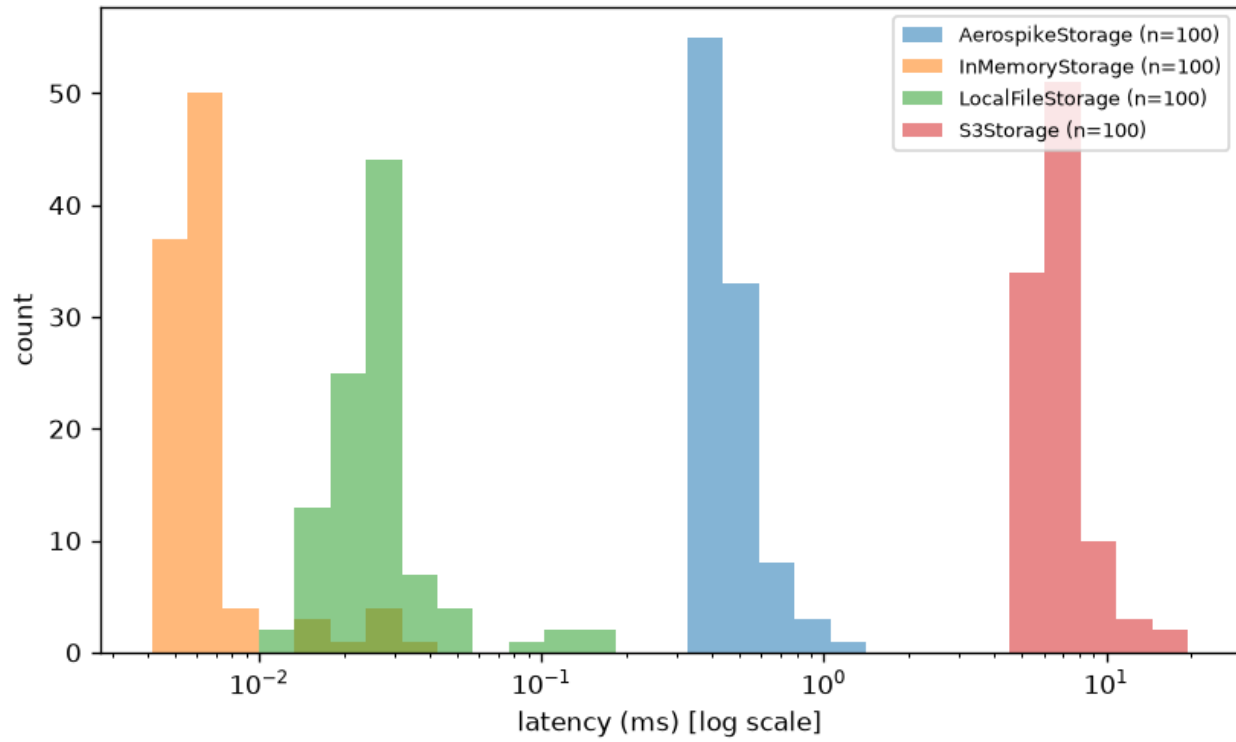


SessionRepository.read_message @ scale=2000



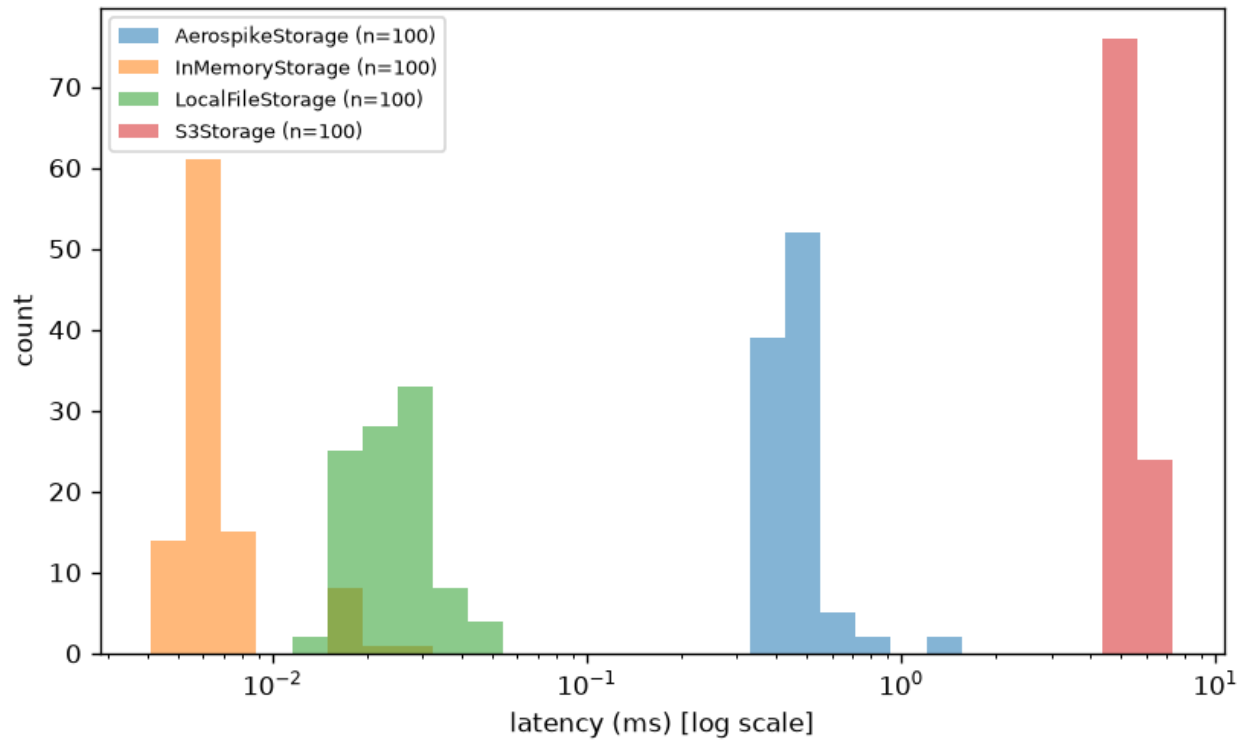
Storage.delete @ scale=100

Storage.delete @ scale=100



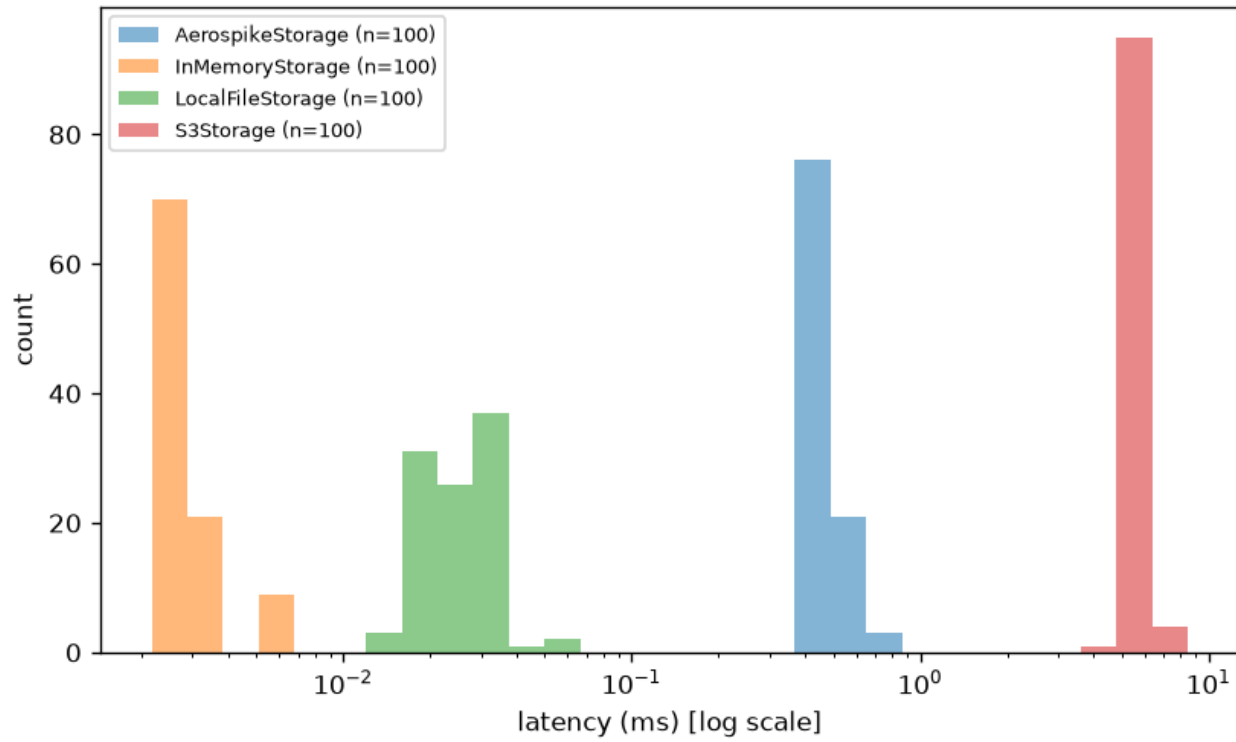
Storage.delete @ scale=500

Storage.delete @ scale=500



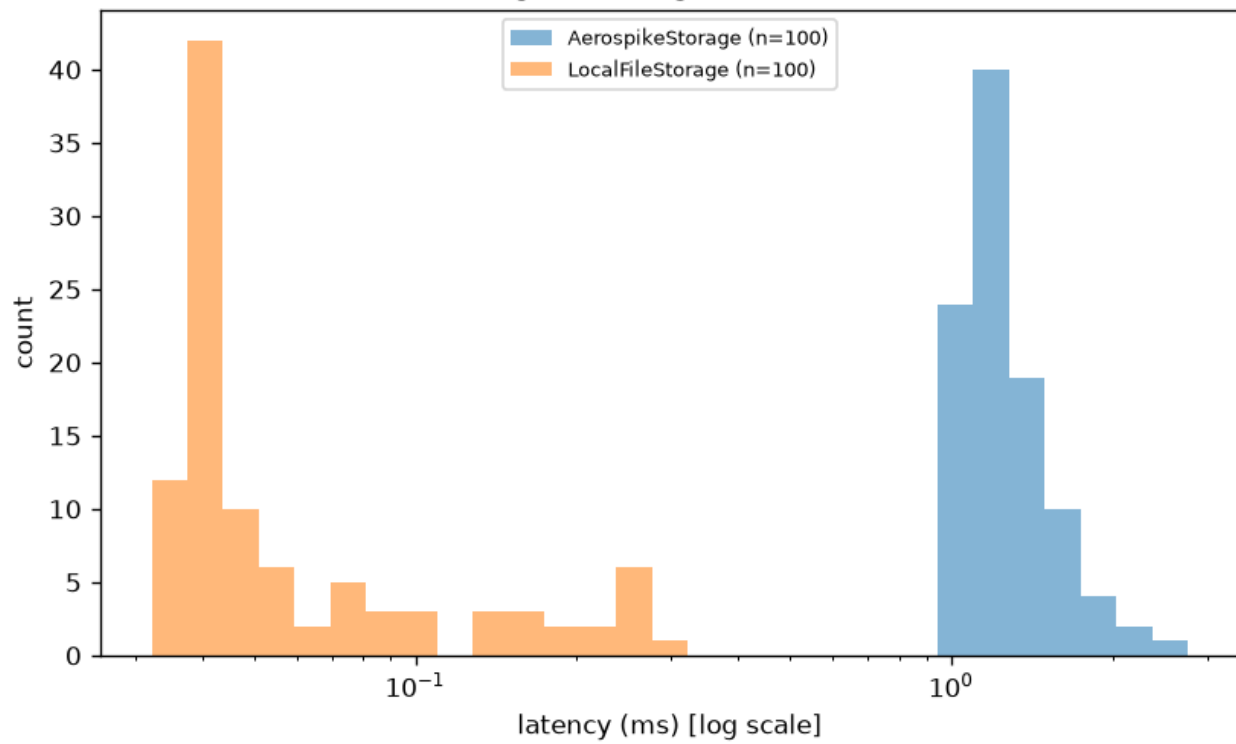
Storage.delete @ scale=2000

Storage.delete @ scale=2000



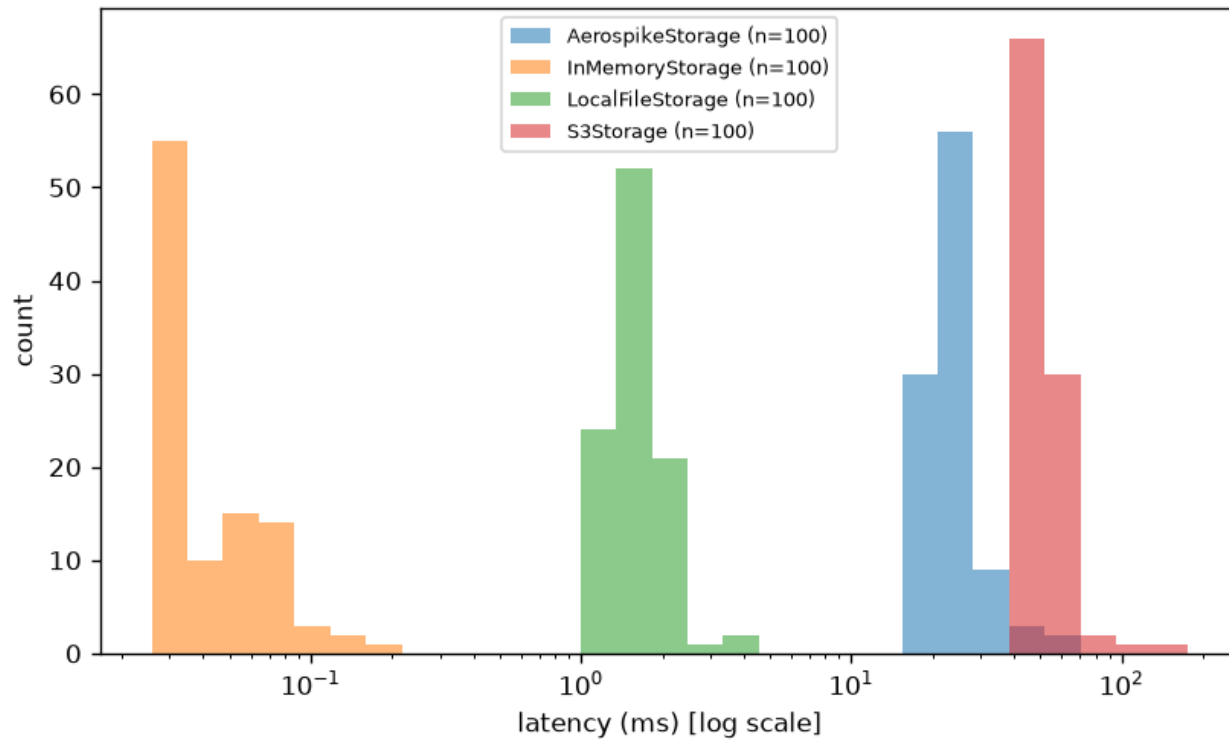
Storage.delete @ scale=20480

Storage.delete @ scale=20480



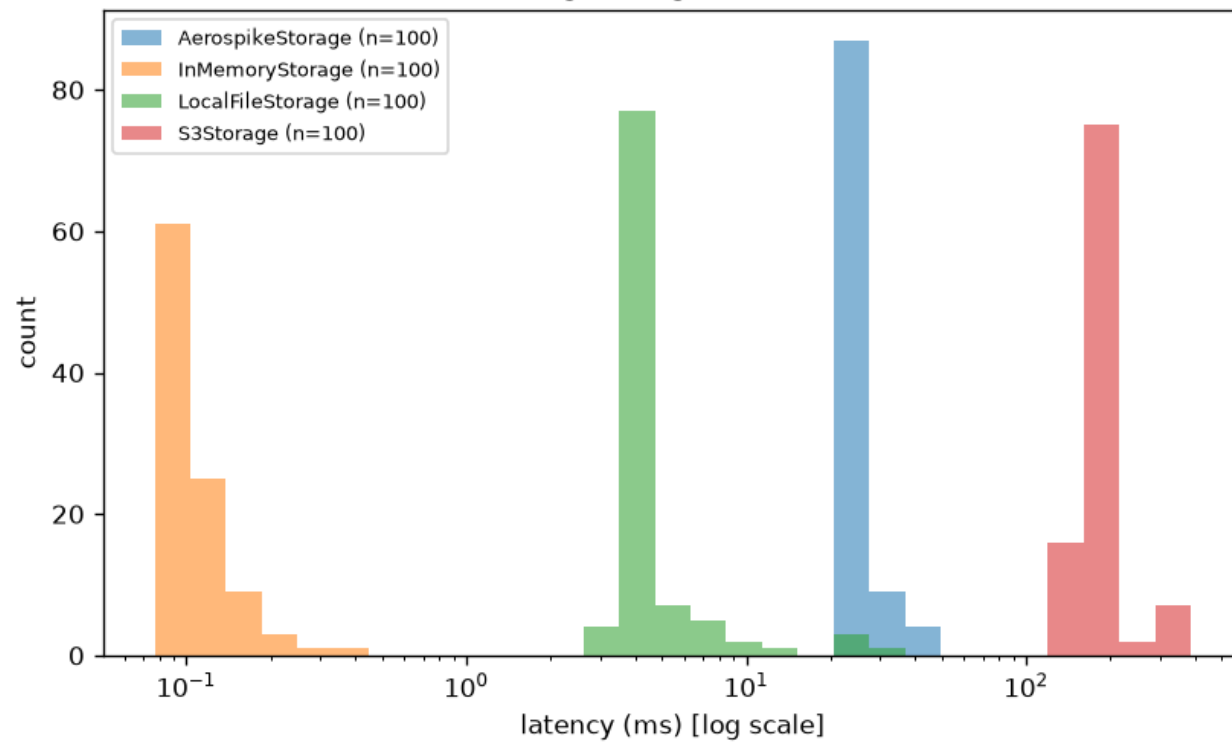
Storage.list @ scale=100

Storage.list @ scale=100



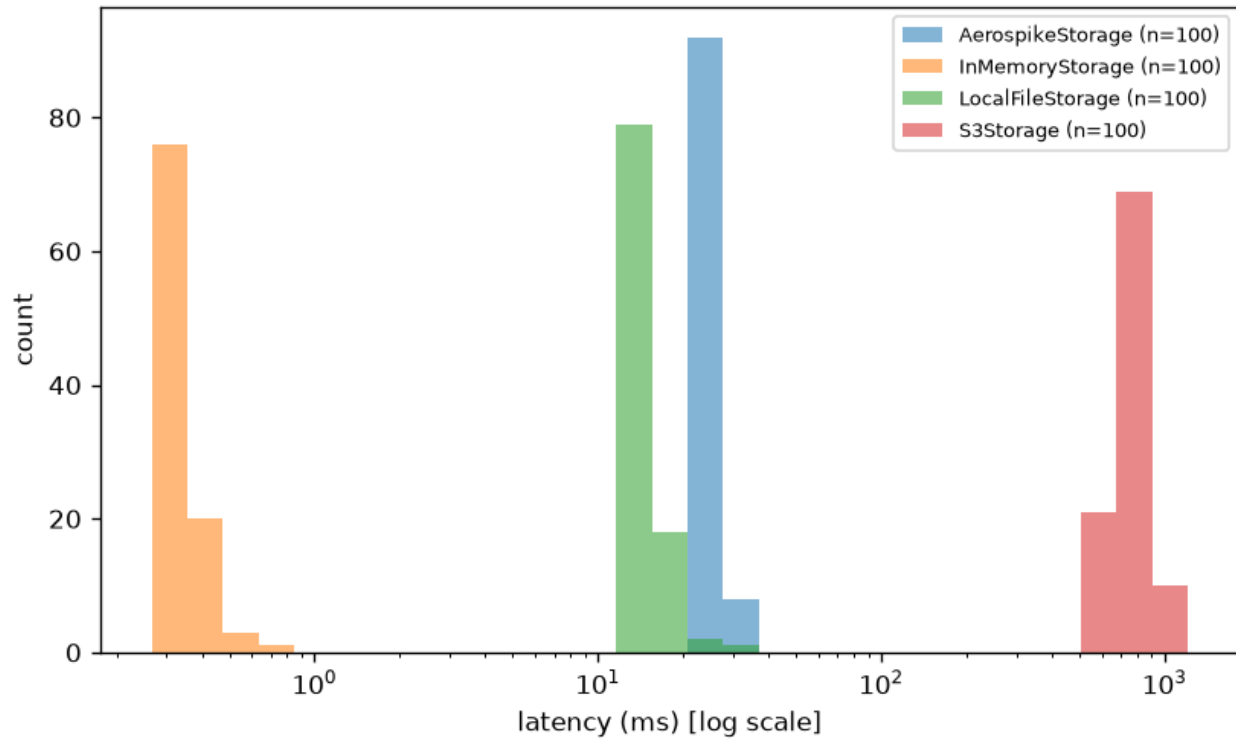
Storage.list @ scale=500

Storage.list @ scale=500



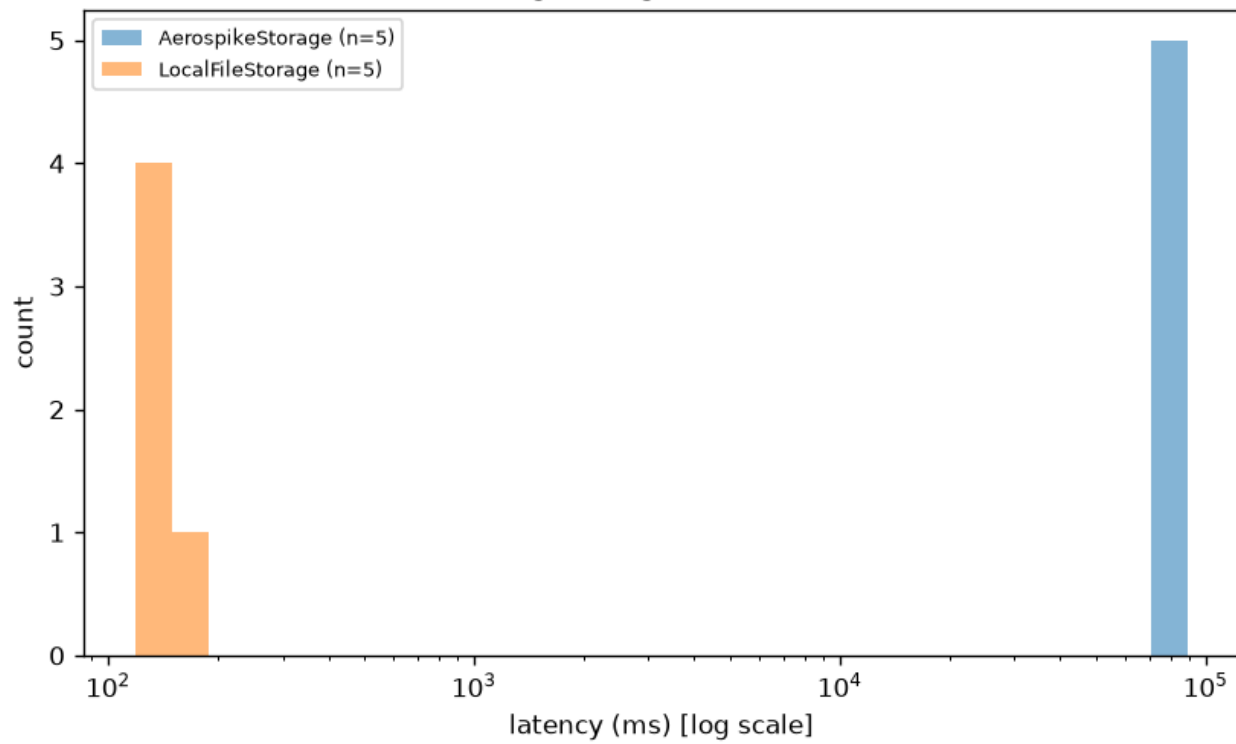
Storage.list @ scale=2000

Storage.list @ scale=2000



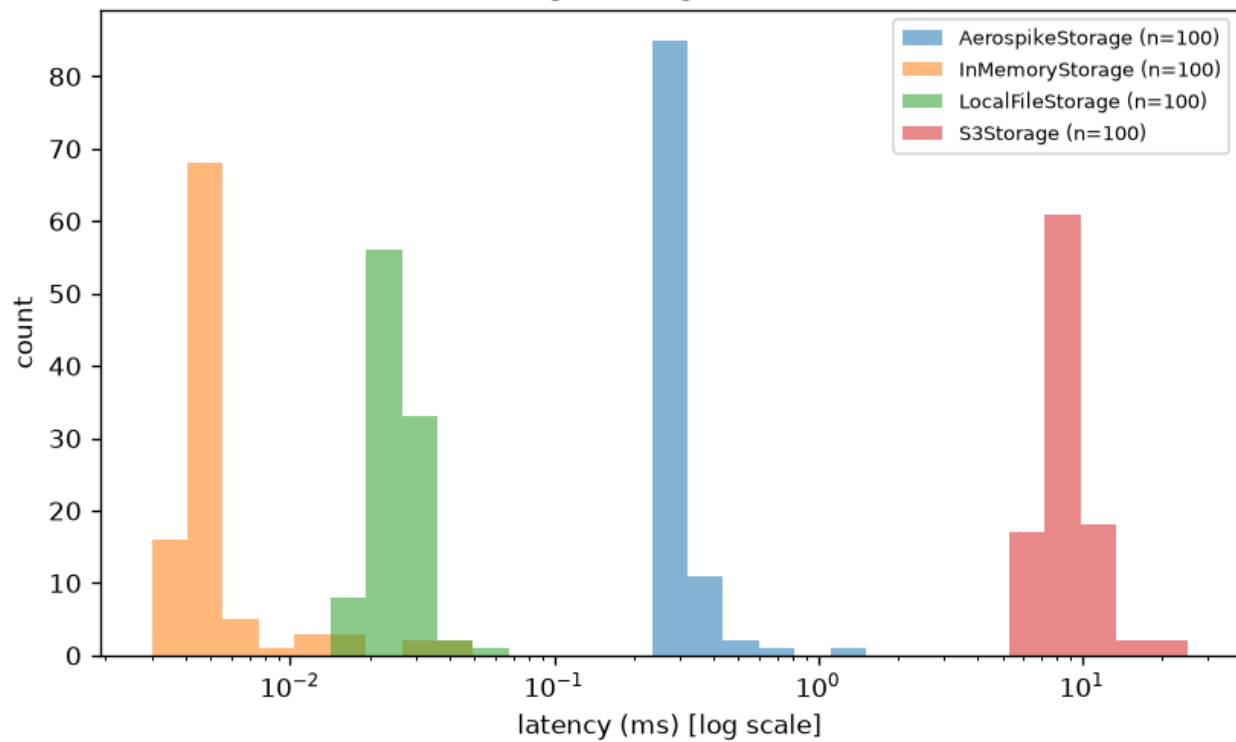
Storage.list @ scale=20480

Storage.list @ scale=20480



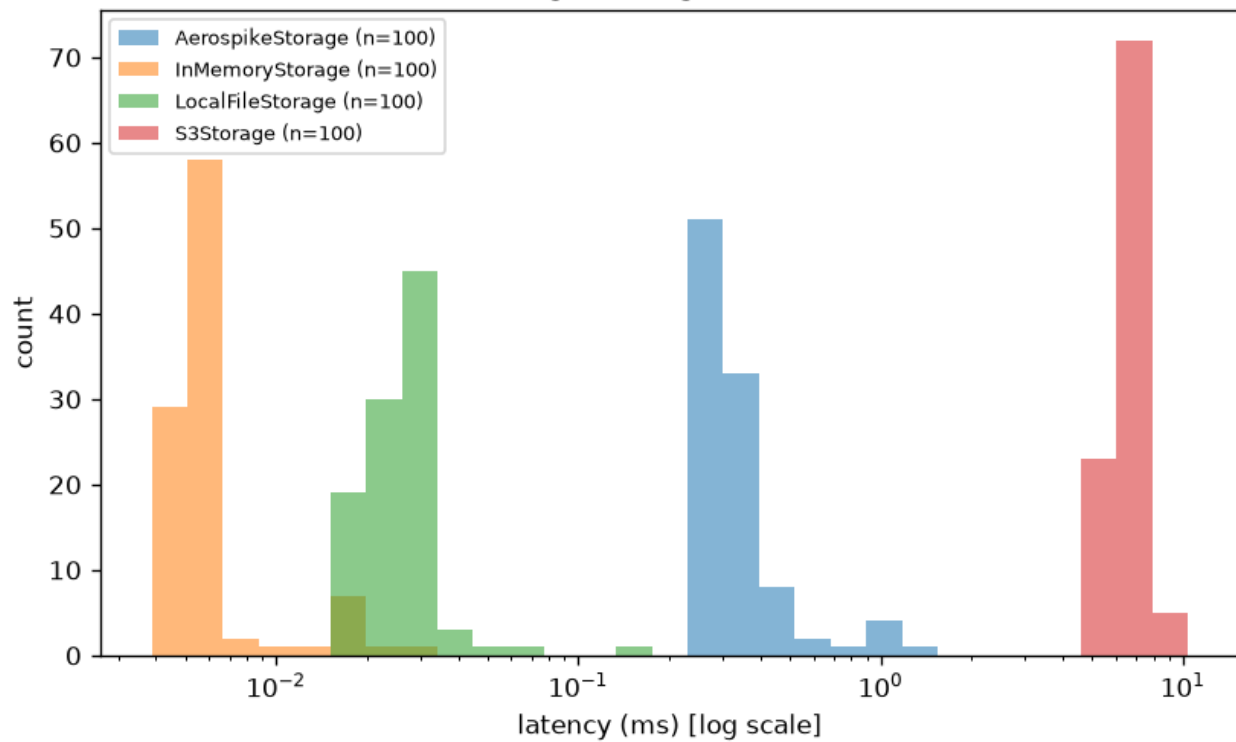
Storage.read @ scale=100

Storage.read @ scale=100



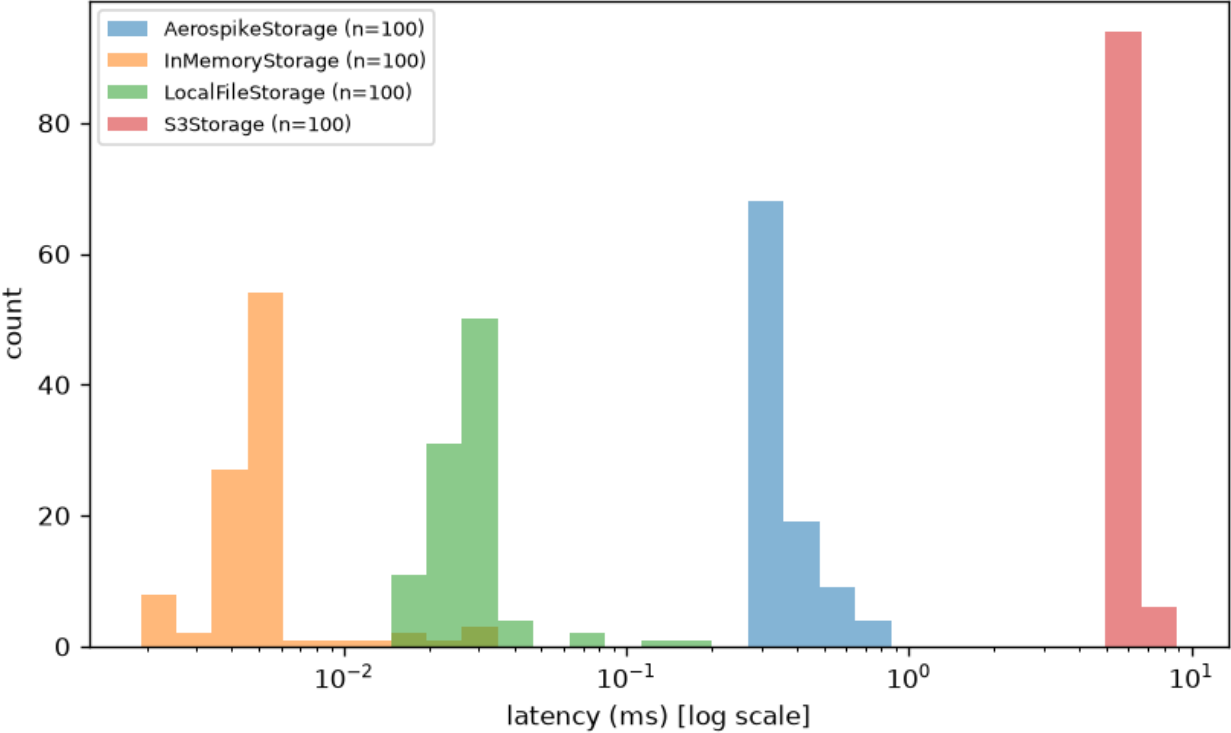
Storage.read @ scale=500

Storage.read @ scale=500



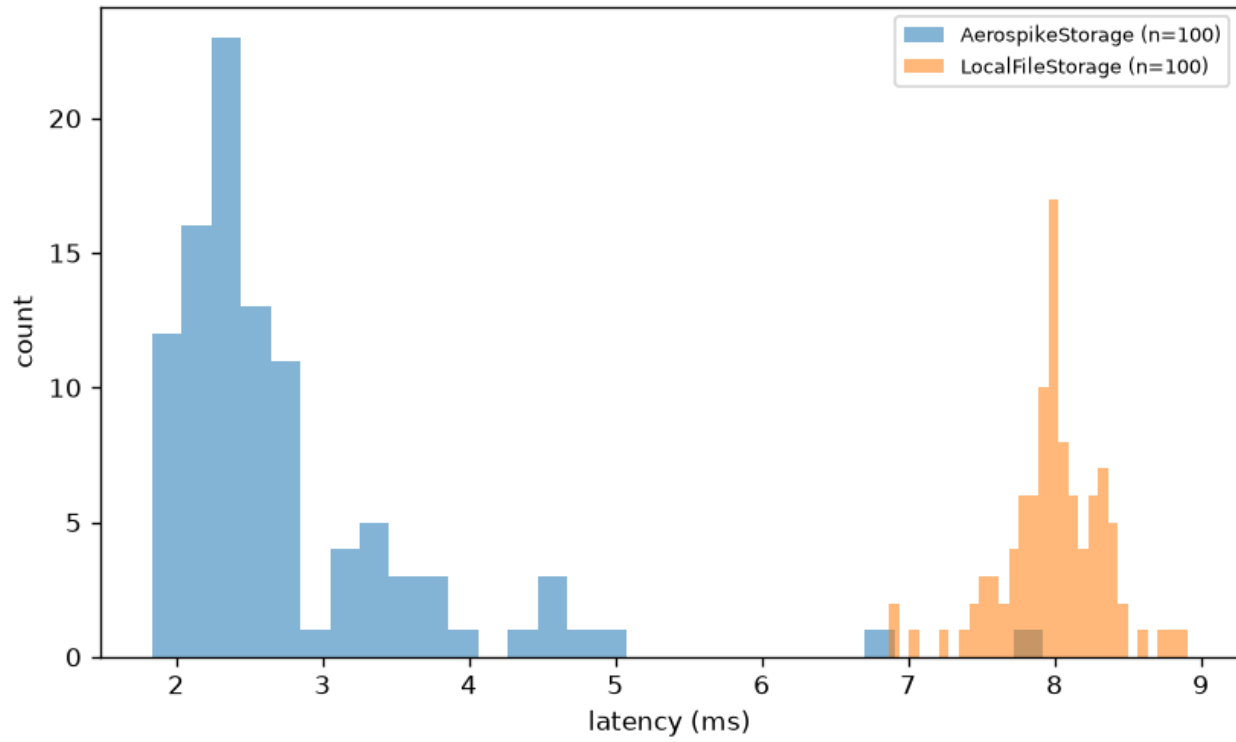
Storage.read @ scale=2000

Storage.read @ scale=2000



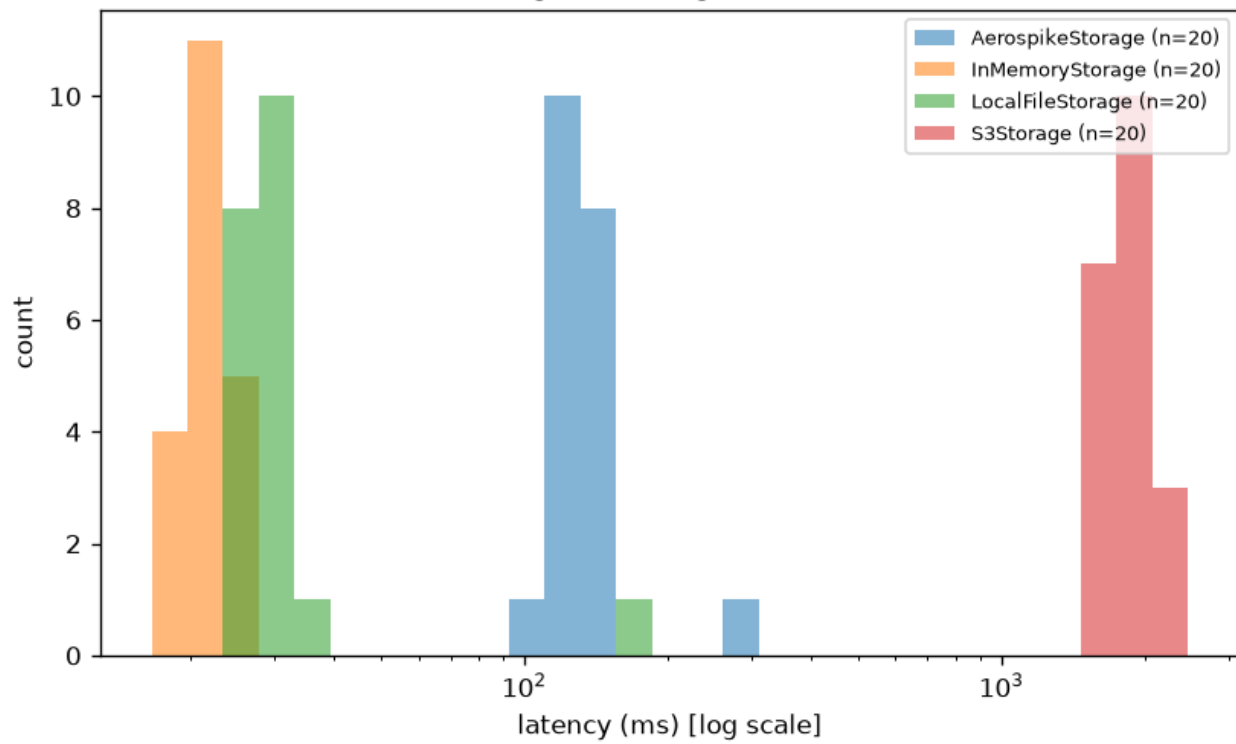
Storage.read @ scale=20480

Storage.read @ scale=20480



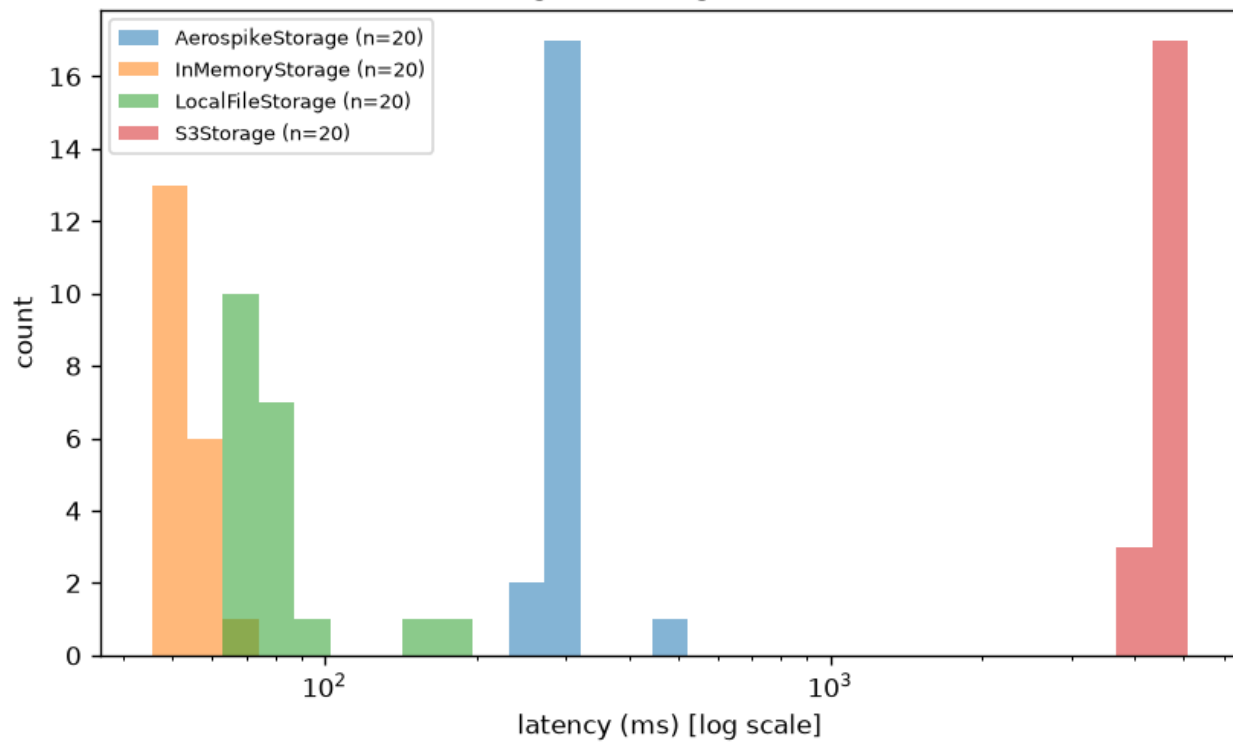
Storage.search @ scale=100

Storage.search @ scale=100



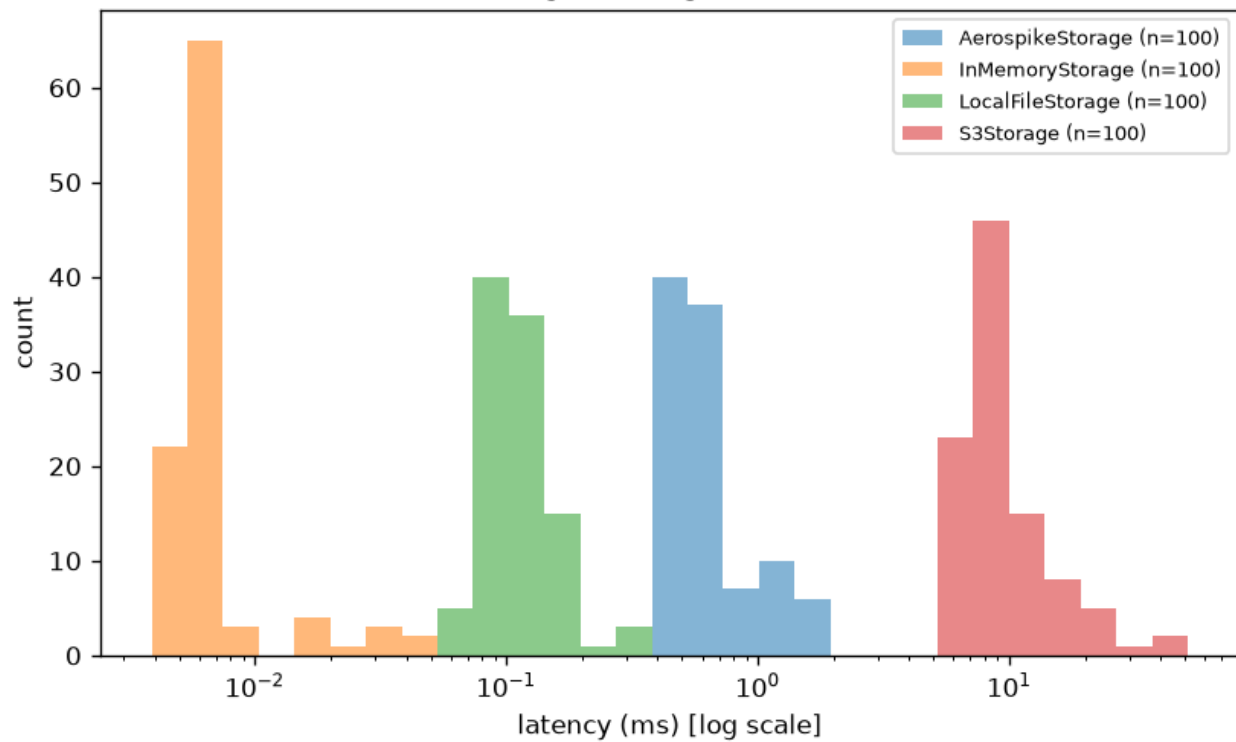
Storage.search @ scale=500

Storage.search @ scale=500



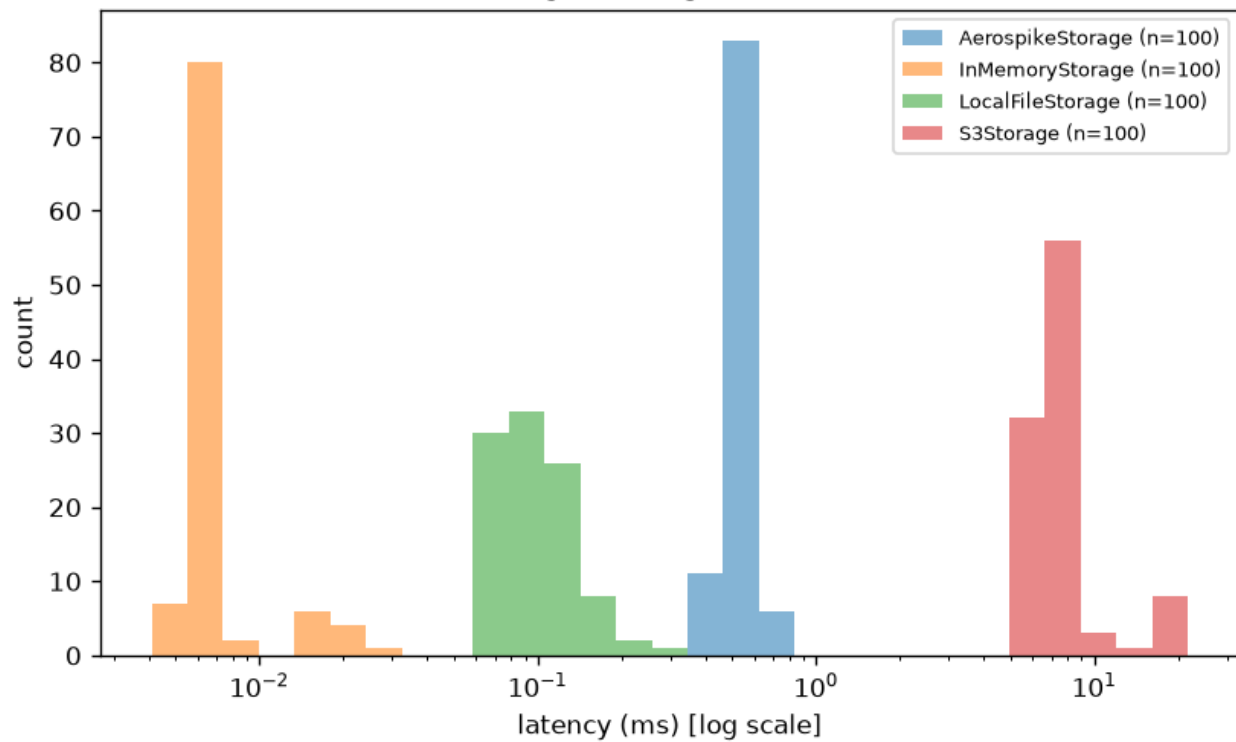
Storage.write @ scale=100

Storage.write @ scale=100



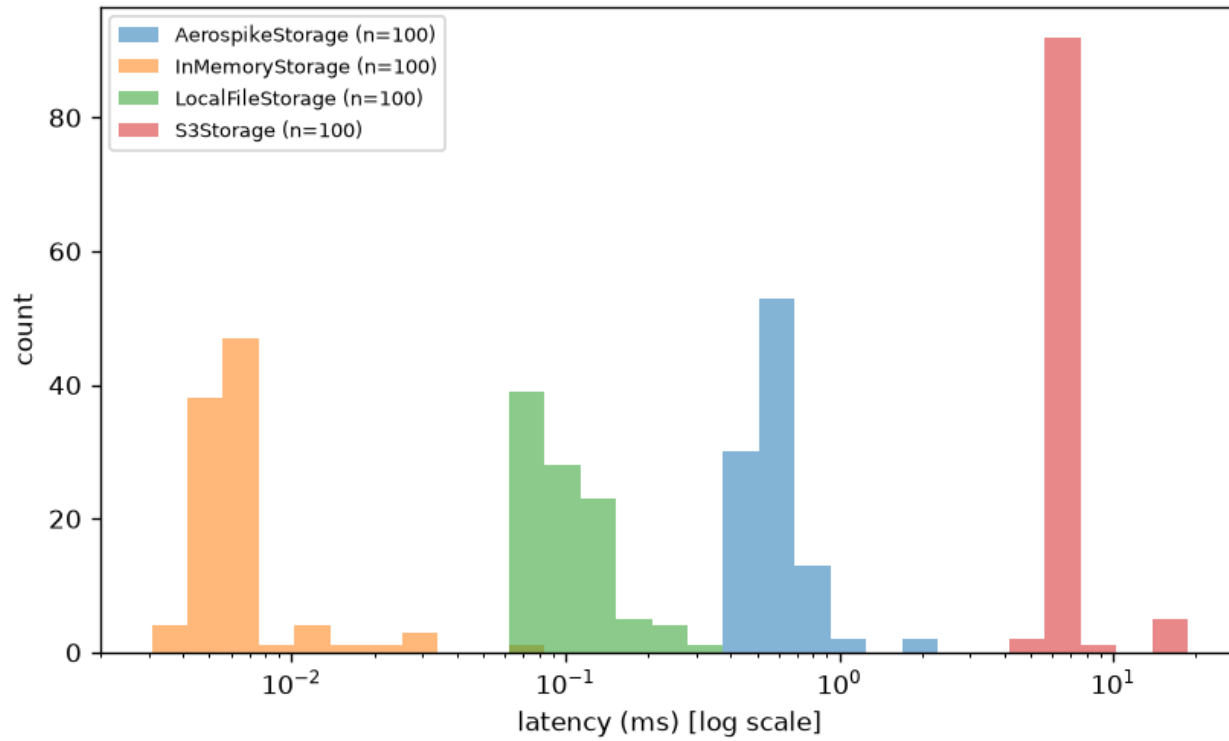
Storage.write @ scale=500

Storage.write @ scale=500

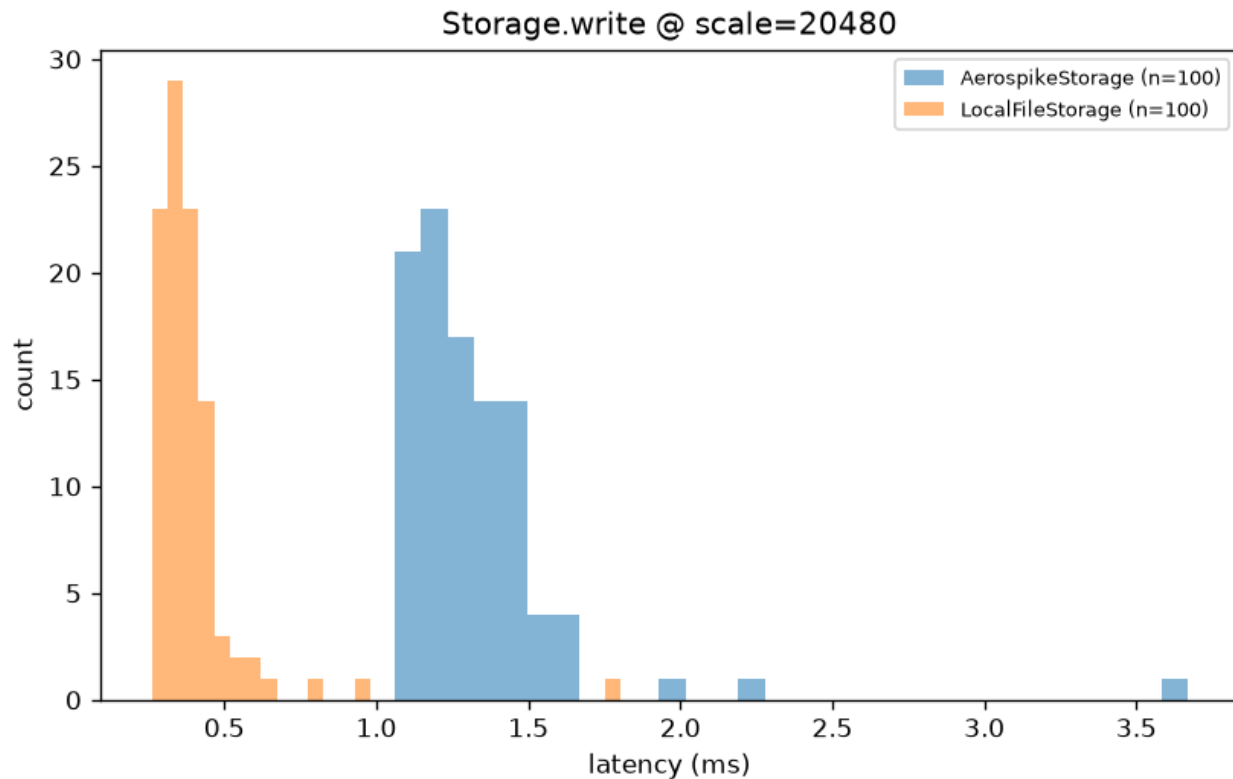


Storage.write @ scale=2000

Storage.write @ scale=2000



Storage.write @ scale=20480



Interpretation

- Storage.list, ranked fastest to slowest at scale=20480:** LocalFileStorage 139.007ms; AerospikeStorage 80818.735ms. Growth from scale=100 to scale=20480: LocalFileStorage 1.643ms@100 -> 139.007ms@20480 (grows sub-linearly, 84.6x); AerospikeStorage 24.237ms@100 -> 80818.735ms@20480 (grows roughly linearly, 3334.5x). AerospikeStorage's `list` filters server-side with a compiled regex expression during a scan (see docs/DESIGN.md), so only matching records are ever serialized back to the client -- the advantage that buys is in *bytes shipped*, not in avoiding the server-side scan itself, and a `scan()` call carries fixed per-call job-setup overhead that a direct directory walk or in-process dict iteration does not pay. On a single local node with small local values, that fixed overhead can dominate small-to-medium key counts; it matters most exactly when scanning-and-shipping unfiltered data would be expensive, i.e. large key counts and/or a real multi-node cluster where network egress (not local disk) is the bottleneck a naive `list()` avoids.
- List_messages, ranked fastest to slowest at scale=2000 messages:** AerospikeSessionManager 438.758ms; FileSessionManager 478.056ms; S3SessionManager 14517.594ms. AerospikeSessionManager never scans to build this result: the agent record's message-count bin gives the exact key range, fetched with one `batch_read` (see docs/DESIGN.md) -- one round trip regardless of how the range is fetched, versus FileSessionManager's directory listing plus one file open per message, and S3SessionManager's per-object or paginated-list calls, both of which scale with message count and, for S3, with real per-request HTTP overhead against the local moto server.
- MemoryStore.search, ranked fastest to slowest at scale=5000 entries:** AerospikeMemoryStore 92.302ms; FileMemoryStore 1437.493ms. Growth from smallest to largest scale tested: AerospikeMemoryStore 5.015ms@50 -> 92.302ms@5000 (grows sub-linearly, 18.4x); FileMemoryStore 9.401ms@50 -> 1437.493ms@5000 (grows roughly linearly, 152.9x).

AerospikeMemoryStore looks up the query token via a list secondary index (`predicates.contains`, see docs/DESIGN.md) -- cost driven by the number of *matching* entries, not store size. FileMemoryStore's default `KeywordSearchStrategy` fetches every entry's content and scores it client-side, so its cost grows with total store size regardless of how many entries actually match; the growth figures above are the direct evidence for that difference. Because every entry matches this particular query, though, the index can't actually narrow anything here either -- see `search_selective` below for the case an index-backed lookup is built for.

- **MemoryStore.search_selective, ranked fastest to slowest at scale=5000 entries:** AerospikeMemoryStore 2.503ms; FileMemoryStore 1458.554ms. Growth from smallest to largest scale tested: AerospikeMemoryStore 3.602ms@50 -> 2.503ms@5000 (roughly flat, 0.7x); FileMemoryStore 9.151ms@50 -> 1458.554ms@5000 (grows roughly linearly, 159.4x). This query matches exactly one store-unique entry regardless of scale -- the case the tok list secondary index (see docs/DESIGN.md) exists for: AerospikeMemoryStore's cost should track the (constant) number of matches, not store size, while FileMemoryStore's `KeywordSearchStrategy` still fetches and scores every entry no matter how selective the query is. Compare this growth against `search` above, where every entry matched: that's the direct evidence for whether the index is actually paying for itself here.
- **Single-key write, ranked fastest to slowest at scale=20480:** LocalFileStorage 0.395ms; AerospikeStorage 1.323ms. **Single-key read:** AerospikeStorage 2.746ms; LocalFileStorage 7.988ms. Every backend maps these to one underlying key lookup (AerospikeStorage: one record put/get; AerospikeSessionManager's equivalent `create_message/read_message` paths are the 'pure key lookups' design choice documented in docs/DESIGN.md). InMemoryStorage has no I/O at all, so it is the floor every persistent backend is compared against, not a production competitor (nothing survives a process restart); the gap between it and every other backend here is the actual cost of durability, not an inefficiency in any of them.

Reproduction

```
./scripts/start_aerospike_ce.sh
uv pip install --python .venv/bin/python -e ".[dev,benchmark]" -e ../harness-sdk/strands-py
.venv/bin/python benchmark/benchmark.py # default 10GB bulk tier
.venv/bin/python benchmark/benchmark.py --bulk-target-gb 100 # once more host disk is available
```

Appendix: full raw statistics

interface	operation	backend	scale	mean (ms)	median (ms)	p95 (ms)	p99 (ms)	throughput (ops/s)	n
MemoryStore	add	AerospikeMemoryStore	50	0.899	0.626	2.613	4.955	1112.5	100
MemoryStore	add	FileMemoryStore	50	0.245	0.198	0.697	0.739	4075.6	100
MemoryStore	add	AerospikeMemoryStore	250	0.670	0.578	0.849	1.879	1493.2	100
MemoryStore	add	FileMemoryStore	250	0.184	0.170	0.286	0.517	5430.2	100
MemoryStore	add	AerospikeMemoryStore	1000	0.628	0.563	0.789	1.737	1591.5	100
MemoryStore	add	FileMemoryStore	1000	0.159	0.167	0.231	0.248	6284.7	100
MemoryStore	add	AerospikeMemoryStore	5000	1.272	1.213	1.679	1.750	786.3	100

interface	operation	backend	scale	mean (ms)	median (ms)	p95 (ms)	p99 (ms)	throughput (ops/s)	n
MemoryStore	add	FileMemoryStore	5000	0.456	0.415	0.648	0.812	2195.0	100
MemoryStore	search	AerospikeMemoryStore	50	5.015	3.999	8.782	11.491	199.4	100
MemoryStore	search	FileMemoryStore	50	9.401	7.170	18.120	48.845	106.4	100
MemoryStore	search	AerospikeMemoryStore	250	6.077	5.743	7.770	9.392	164.5	100
MemoryStore	search	FileMemoryStore	250	21.463	14.893	25.133	91.880	46.6	100
MemoryStore	search	AerospikeMemoryStore	1000	11.667	11.436	13.176	14.658	85.7	100
MemoryStore	search	FileMemoryStore	1000	61.651	48.201	231.722	278.455	16.2	100
MemoryStore	search	AerospikeMemoryStore	5000	92.302	89.053	102.002	114.058	10.8	100
MemoryStore	search	FileMemoryStore	5000	1437.493	1424.658	1672.416	1724.360	0.7	100
MemoryStore	search_selective	AerospikeMemoryStore	50	3.602	2.763	6.843	9.019	277.6	100
MemoryStore	search_selective	FileMemoryStore	50	9.151	8.108	14.211	21.035	109.3	100
MemoryStore	search_selective	AerospikeMemoryStore	250	2.338	2.254	3.018	3.351	427.6	100
MemoryStore	search_selective	FileMemoryStore	250	17.173	14.590	20.947	28.461	58.2	100
MemoryStore	search_selective	AerospikeMemoryStore	1000	2.318	2.215	2.908	3.877	431.3	100
MemoryStore	search_selective	FileMemoryStore	1000	59.799	45.311	238.391	257.565	16.7	100
MemoryStore	search_selective	AerospikeMemoryStore	5000	2.503	2.447	2.898	3.390	399.5	100
MemoryStore	search_selective	FileMemoryStore	5000	1458.554	1441.358	1684.451	1735.545	0.7	100
SessionRepository	create_message	AerospikeSessionManager	20	0.414	0.370	0.674	0.700	2413.3	100
SessionRepository	create_message	FileSessionManager	20	0.348	0.284	0.843	0.935	2870.8	100
SessionRepository	create_message	S3SessionManager	20	6.858	6.231	9.039	16.582	145.8	100
SessionRepository	create_message	AerospikeSessionManager	100	0.390	0.367	0.488	0.614	2567.2	100

interface	operation	backend	scale	mean (ms)	median (ms)	p95 (ms)	p99 (ms)	throughput (ops/s)	n
SessionRepository	create_message	FileSessionManager	100	0.224	0.225	0.317	0.357	4454.5	100
SessionRepository	create_message	S3SessionManager	100	6.801	6.230	7.964	17.947	147.0	100
SessionRepository	create_message	AerospikeSessionManager	100	0.434	0.379	0.600	1.223	2303.8	100
SessionRepository	create_message	FileSessionManager	500	0.235	0.230	0.335	0.474	4261.9	100
SessionRepository	create_message	S3SessionManager	500	6.801	6.037	16.214	16.933	147.0	100
SessionRepository	create_message	AerospikeSessionManager	2000	0.480	0.451	0.658	0.739	2081.8	100
SessionRepository	create_message	FileSessionManager	2000	0.313	0.308	0.415	0.539	3195.3	100
SessionRepository	create_message	S3SessionManager	2000	6.583	6.135	7.693	16.843	151.9	100
SessionRepository	delete_session	AerospikeSessionManager	20	1.256	1.030	2.422	3.013	795.9	20
SessionRepository	delete_session	FileSessionManager	20	0.888	0.679	0.956	4.153	1125.6	20
SessionRepository	delete_session	S3SessionManager	20	34.232	32.319	48.475	53.655	29.2	20
SessionRepository	list_messages	AerospikeSessionManager	20	28.576	26.737	41.652	49.583	35.0	100
SessionRepository	list_messages	FileSessionManager	20	34.185	29.775	46.784	119.997	29.3	100
SessionRepository	list_messages	S3SessionManager	20	940.153	922.438	1098.506	1175.723	1.1	100
SessionRepository	list_messages	AerospikeSessionManager	100	38.154	37.472	43.373	45.842	26.2	100
SessionRepository	list_messages	FileSessionManager	100	44.540	44.304	50.254	51.766	22.5	100
SessionRepository	list_messages	S3SessionManager	100	1556.385	1537.396	1804.325	1912.562	0.6	100
SessionRepository	list_messages	AerospikeSessionManager	500	122.299	120.019	143.559	156.107	8.2	100
SessionRepository	list_messages	FileSessionManager	500	126.752	125.742	145.246	155.370	7.9	100
SessionRepository	list_messages	S3SessionManager	500	4052.012	4034.731	4340.192	4450.623	0.2	100
SessionRepository	list_messages	AerospikeSessionManager	2000	438.758	415.693	607.964	641.858	2.3	100

interface	operation	backend	scale	mean (ms)	median (ms)	p95 (ms)	p99 (ms)	throughput (ops/s)	n
SessionRepository	list_messages	FileSessionManager	2000	478.056	469.171	548.866	593.683	2.1	100
SessionRepository	list_messages	S3SessionManager	2000	14517.594	14371.878	15965.558	17088.851	0.1	100
SessionRepository	read_message	AerospikeSessionManager	20	0.431	0.393	0.677	0.768	2321.1	100
SessionRepository	read_message	FileSessionManager	20	0.417	0.368	0.874	0.946	2399.9	100
SessionRepository	read_message	S3SessionManager	20	5.766	5.637	6.601	7.504	173.4	100
SessionRepository	read_message	AerospikeSessionManager	100	0.385	0.357	0.510	0.524	2594.6	100
SessionRepository	read_message	FileSessionManager	100	0.269	0.257	0.395	0.486	3718.5	100
SessionRepository	read_message	S3SessionManager	100	5.966	5.859	6.945	7.163	167.6	100
SessionRepository	read_message	AerospikeSessionManager	500	0.411	0.370	0.614	0.721	2430.8	100
SessionRepository	read_message	FileSessionManager	500	0.258	0.233	0.365	0.483	3871.5	100
SessionRepository	read_message	S3SessionManager	500	6.441	6.110	8.124	9.513	155.2	100
SessionRepository	read_message	AerospikeSessionManager	2000	0.450	0.404	0.639	0.914	2221.7	100
SessionRepository	read_message	FileSessionManager	2000	0.295	0.266	0.419	0.534	3394.9	100
SessionRepository	read_message	S3SessionManager	2000	6.097	5.961	7.144	8.623	164.0	100
Storage	delete	AerospikeStorage	100	0.482	0.433	0.658	1.027	2075.0	100
Storage	delete	InMemoryStorage	100	0.007	0.006	0.019	0.031	133498.5	100
Storage	delete	LocalFileStorage	100	0.031	0.026	0.057	0.155	32664.0	100
Storage	delete	S3Storage	100	7.009	6.537	10.137	14.747	142.7	100
Storage	delete	AerospikeStorage	500	0.477	0.436	0.709	1.274	2094.8	100
Storage	delete	InMemoryStorage	500	0.007	0.006	0.018	0.020	134242.2	100
Storage	delete	LocalFileStorage	500	0.025	0.024	0.035	0.047	40811.5	100

interface	operation	backend	scale	mean (ms)	median (ms)	p95 (ms)	p99 (ms)	throughput (ops/s)	n
Storage	delete	S3Storage	500	5.438	5.296	6.230	6.452	183.9	100
Storage	delete	AerospikeStorage	2000	0.463	0.442	0.583	0.662	2160.8	100
Storage	delete	InMemoryStorage	2000	0.003	0.003	0.006	0.006	328404.0	100
Storage	delete	LocalFileStorage	2000	0.026	0.028	0.035	0.052	38261.0	100
Storage	delete	S3Storage	2000	5.398	5.289	6.228	7.663	185.3	100
Storage	delete	AerospikeStorage	20480	1.294	1.215	1.969	2.154	773.1	100
Storage	delete	LocalFileStorage	20480	0.075	0.042	0.247	0.260	13372.8	100
Storage	list	AerospikeStorage	100	24.237	21.651	38.146	52.275	41.3	100
Storage	list	InMemoryStorage	100	0.046	0.031	0.099	0.120	21604.9	100
Storage	list	LocalFileStorage	100	1.643	1.508	2.185	3.905	608.6	100
Storage	list	S3Storage	100	51.956	49.337	66.103	112.019	19.2	100
Storage	list	AerospikeStorage	500	23.833	21.823	35.657	41.040	42.0	100
Storage	list	InMemoryStorage	500	0.111	0.088	0.185	0.291	9049.3	100
Storage	list	LocalFileStorage	500	5.214	3.800	11.300	26.387	191.8	100
Storage	list	S3Storage	500	185.132	172.671	295.079	334.927	5.4	100
Storage	list	AerospikeStorage	2000	26.950	26.685	28.011	31.001	37.1	100
Storage	list	InMemoryStorage	2000	0.340	0.319	0.440	0.525	2939.2	100
Storage	list	LocalFileStorage	2000	14.441	13.587	19.545	24.717	69.2	100
Storage	list	S3Storage	2000	750.907	719.432	922.180	996.306	1.3	100
Storage	list	AerospikeStorage	20480	80818.735	80819.588	80820.254	80820.351	0.0	5
Storage	list	LocalFileStorage	20480	139.007	136.103	148.670	149.467	7.2	5

interface	operation	backend	scale	mean (ms)	median (ms)	p95 (ms)	p99 (ms)	throughput (ops/s)	n
Storage	read	AerospikeStorage	100	0.311	0.282	0.435	0.622	3219.4	100
Storage	read	InMemoryStorage	100	0.007	0.005	0.015	0.046	153588.1	100
Storage	read	LocalFileStorage	100	0.026	0.025	0.032	0.045	38939.0	100
Storage	read	S3Storage	100	9.019	8.279	12.685	22.519	110.9	100
Storage	read	AerospikeStorage	500	0.356	0.301	0.729	1.086	2805.5	100
Storage	read	InMemoryStorage	500	0.007	0.006	0.017	0.020	146258.7	100
Storage	read	LocalFileStorage	500	0.027	0.026	0.036	0.063	37512.3	100
Storage	read	S3Storage	500	6.518	6.366	7.740	8.560	153.4	100
Storage	read	AerospikeStorage	2000	0.357	0.307	0.583	0.708	2800.0	100
Storage	read	InMemoryStorage	2000	0.006	0.005	0.019	0.026	172692.5	100
Storage	read	LocalFileStorage	2000	0.030	0.027	0.038	0.140	33561.6	100
Storage	read	S3Storage	2000	6.011	5.941	6.733	7.647	166.4	100
Storage	read	AerospikeStorage	20480	2.746	2.428	4.586	6.773	364.2	100
Storage	read	LocalFileStorage	20480	7.988	8.002	8.453	8.832	125.2	100
Storage	search	AerospikeStorage	100	137.794	130.660	161.045	259.672	7.3	20
Storage	search	InMemoryStorage	100	21.222	20.489	24.212	25.477	47.1	20
Storage	search	LocalFileStorage	100	35.856	29.158	40.290	149.042	27.9	20
Storage	search	S3Storage	100	1824.754	1794.141	2133.117	2213.173	0.5	20
Storage	search	AerospikeStorage	500	298.955	290.481	326.686	430.924	3.3	20
Storage	search	InMemoryStorage	500	53.720	52.387	59.344	64.573	18.6	20
Storage	search	LocalFileStorage	500	82.939	73.352	164.441	168.242	12.1	20

interface	operation	backend	scale	mean (ms)	median (ms)	p95 (ms)	p99 (ms)	throughput (ops/s)	n
Storage	search	S3Storage	500	4442.390	4448.250	4620.981	4631.224	0.2	20
Storage	write	AerospikeStorage	100	0.698	0.559	1.413	1.893	1431.8	100
Storage	write	InMemoryStorage	100	0.008	0.006	0.021	0.040	128463.2	100
Storage	write	LocalFileStorage	100	0.118	0.105	0.182	0.364	8458.8	100
Storage	write	S3Storage	100	10.670	8.497	20.082	42.953	93.7	100
Storage	write	AerospikeStorage	500	0.510	0.486	0.628	0.797	1962.0	100
Storage	write	InMemoryStorage	500	0.008	0.007	0.018	0.023	127099.4	100
Storage	write	LocalFileStorage	500	0.104	0.093	0.179	0.194	9648.6	100
Storage	write	S3Storage	500	7.997	7.163	16.650	18.320	125.0	100
Storage	write	AerospikeStorage	2000	0.609	0.546	0.814	1.738	1641.7	100
Storage	write	InMemoryStorage	2000	0.007	0.006	0.016	0.033	135745.6	100
Storage	write	LocalFileStorage	2000	0.106	0.089	0.173	0.260	9451.0	100
Storage	write	S3Storage	2000	6.798	6.261	9.423	16.545	147.1	100
Storage	write	AerospikeStorage	20480	1.323	1.249	1.631	2.234	755.9	100
Storage	write	LocalFileStorage	20480	0.395	0.359	0.594	0.953	2531.9	100